Project Source Code

Total source files: 16

app.py

```
"""
app.py - Early Diabetes Risk Screening for adult women and men (Streamlit)

Run:  streamlit run app.py        (after `python train.py` has created ./models)

Two models are served:
  * screening - no blood test needed (sex, age, BMI, hypertension, heart disease,
    smoking)
  * full      - adds HbA1c and blood glucose; used when the person has lab results

Each uses the exact preprocessing code and parameters learned in training
(imputation_params.joblib -> ClinicalPreprocessor -> scaler.joblib ->
    best_model.joblib)
and flags "High Risk" at its own recall-tuned threshold (model_metadata.json).

EDUCATIONAL TOOL ONLY - not a diagnostic device.
"""
from __future__ import annotations

import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st

from preprocessing import (
    BMI_PLACEHOLDER,
    FEATURE_GROUPS,
    FEATURE_SETS,
    ClinicalPreprocessor,
)

MODELS_DIR = Path(__file__).parent / "models"

st.set_page_config(page_title="Early Diabetes Risk Screening", page_icon="■",
    layout="wide")


# ---------------------------------------------------------------------------- #
# Artifacts
# ---------------------------------------------------------------------------- #
def positive_class_shap(values) -> np.ndarray:
    if isinstance(values, list):
        values = values[1]
    values = np.asarray(values)
    return values[:, :, 1] if values.ndim == 3 else values


@st.cache_resource(show_spinner="Loading models...")
def load_artifacts():
    """Load model, scaler, imputation parameters, metadata and SHAP explainer per
    feature set."""
    needed = ["best_model.joblib", "scaler.joblib", "imputation_params.joblib",
    "model_metadata.json"]
    bundles = {}
```

```
    for fs in FEATURE_SETS:
        d = MODELS_DIR / fs
        if not all((d / f).exists() for f in needed):
            return None
        model = joblib.load(d / "best_model.joblib")
        explainer = None
        if not hasattr(model, "coef_"):  # tree model -> SHAP explainer (built once)
            import shap

            explainer = shap.TreeExplainer(model)
        bundles[fs] = {
            "model": model,
            "scaler": joblib.load(d / "scaler.joblib"),
            "prep": ClinicalPreprocessor.from_params_dict(joblib.load(d /
    "imputation_params.joblib")),
            "meta": json.loads((d / "model_metadata.json").read_text()),
            "explainer": explainer,
        }
    return bundles


bundles = load_artifacts()
if bundles is None:
    st.error("Model files not found in `./models`. Run `python train.py` first, then
    reload this page.")
    st.stop()


# ------------------------------------------------------------------------ #
# Clinical reference ranges (shown under each input)
# ------------------------------------------------------------------------ #
INF = float("inf")
RANGES = {
    "age": [(35, 0, "Lower age-related risk (< 35)"), (45, 1, "Rising risk
    (35-44)"),
            (INF, 2, "Higher risk (45+)")],
    "bmi": [(18.5, 1, "Underweight (< 18.5)"), (25.0, 0, "Healthy (18.5-24.9)"),
            (30.0, 1, "Overweight (25-29.9)"), (INF, 2, "Obese (30+)")],
    "HbA1c_level": [(5.7, 0, "Normal (< 5.7%)"), (6.5, 1, "Prediabetes range
    (5.7-6.4%)"),
                    (INF, 2, "Diabetes range (6.5%+)")],
    "blood_glucose_level": [(100, 0, "Normal if fasting (< 100)"), (140, 1, "Raised
    (100-139)"),
                            (200, 1, "High (140-199)"), (INF, 2, "Very high
    (200+)")],
}
LEVEL_ICON = {0: "■", 1: "■", 2: "■"}


def classify(feature: str, value: float) -> tuple[int, str]:
    for upper, level, label in RANGES[feature]:
        if value < upper:
            return level, label
    return 2, RANGES[feature][-1][2]


# ------------------------------------------------------------------------ #
# Prediction + explanation
# ------------------------------------------------------------------------ #
def predict(bundle: dict, patient: dict):
```

```
    """Raw patient dict -> (probability, cleaned features, scaled features)."""
    raw = pd.DataFrame([patient])
    cleaned = bundle["prep"].transform(raw)
    scaled = pd.DataFrame(np.asarray(bundle["scaler"].transform(cleaned)),
    columns=cleaned.columns)
    return float(bundle["model"].predict_proba(scaled)[0, 1]), cleaned, scaled


def explain(bundle: dict, scaled: pd.DataFrame) -> tuple[pd.Series, str]:
    """Per-factor contribution relative to an average patient, grouped
    clinically."""
    model = bundle["model"]
    if hasattr(model, "coef_"):
        contrib = pd.Series(model.coef_[0] * scaled.iloc[0].to_numpy(),
    index=scaled.columns)
        unit = "log-odds"
    else:
        contrib =
    pd.Series(positive_class_shap(bundle["explainer"].shap_values(scaled))[0],
                          index=scaled.columns)
        # tree ensembles from scikit-learn explain probability; XGBoost explains
    log-odds
        unit = "log-odds" if type(model).__name__.startswith("XGB") else
    "probability"
    return contrib.groupby(lambda f: FEATURE_GROUPS.get(f, f)).sum(), unit


# ---------------------------------------------------------------------------- #
# Visual components
# ---------------------------------------------------------------------------- #
def risk_badge(high: bool) -> None:
    if high:
        style = "background:#fde8e8;color:#7a1414;border:2px solid #e34948;"
        text, icon = "High Risk of Diabetes", "■■"
    else:
        style = "background:#e3f6ec;color:#0b4f33;border:2px solid #1baf7a;"
        text, icon = "Low Risk of Diabetes", "■"
    st.markdown(
        f"<div style='{style}border-radius:12px;padding:18px
    22px;font-size:1.6rem;font-weight:700;"
        f"text-align:center;'>{icon} {text}</div>",
        unsafe_allow_html=True,
    )


def probability_gauge(prob: float, high: bool, threshold: float) -> go.Figure:
    top = max(25.0, min(100.0, float(np.ceil(max(prob * 100, threshold * 100) * 1.6
    / 5) * 5)))
    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=prob * 100,
        number={"suffix": "%", "valueformat": ".1f", "font": {"size": 44}},
        title={"text": "Estimated probability of diabetes"},
        gauge={
            "axis": {"range": [0, top], "ticksuffix": "%"},
            "bar": {"color": "#c53030" if high else "#1a7f5a", "thickness": 0.28},
            "steps": [
                {"range": [0, threshold * 100], "color": "rgba(27,175,122,0.30)"},
                {"range": [threshold * 100, top], "color": "rgba(227,73,72,0.30)"},
            ],
```

```
            "threshold": {"line": {"color": "#111111", "width": 4}, "thickness":
    0.85,
                          "value": threshold * 100},
        },
    ))
    fig.update_layout(height=300, margin=dict(l=30, r=30, t=60, b=10))
    return fig


def drivers_chart(contrib: pd.Series, unit: str) -> go.Figure:
    top =
    contrib.reindex(contrib.abs().sort_values(ascending=False).index).head(6).iloc[:
    :-1]
    colors = ["#e34948" if v > 0 else "#2a78d6" for v in top]
    fmt = "{:+.2f}" if unit == "log-odds" else "{:+.1%}"
    fig = go.Figure(go.Bar(
        x=top.values, y=top.index, orientation="h", marker_color=colors,
        text=[fmt.format(v) for v in top.values], textposition="outside",
        hovertemplate="%{y}: %{x:+.3f}<extra></extra>",
    ))
    pad = max(0.05 if unit != "log-odds" else 0.3, float(np.abs(top.values).max()) *
    0.3)
    fig.update_layout(
        height=300, margin=dict(l=10, r=10, t=10, b=40), showlegend=False,
        xaxis=dict(title=f"← lowers risk | raises risk →  ({unit}, vs. an average
    patient)",
                   zeroline=True, range=[min(top.values.min(), 0) - pad,
    max(top.values.max(), 0) + pad]),
        yaxis=dict(title=None),
    )
    return fig


# ---------------------------------------------------------------------------- #
# Clinical advice
# ---------------------------------------------------------------------------- #
def personalised_tips(p: dict, has_labs: bool, bmi_unknown: bool) -> list[str]:
    tips = []
    if has_labs:
        a1c, glu = p["HbA1c_level"], p["blood_glucose_level"]
        if a1c >= 6.5:
            tips.append("**HbA1c:** 6.5% or higher is in the diabetes range. A
    repeat or confirmatory test "
                        "ordered by a clinician is the next step.")
        elif a1c >= 5.7:
            tips.append("**HbA1c:** 5.7-6.4% is the prediabetes range. This stage is
    often reversible with "
                        "lifestyle change - ask about a prevention programme.")
        if glu >= 200:
            tips.append("**Blood glucose:** 200 mg/dL or higher is in the diabetes
    range, especially with symptoms "
                        "such as thirst or frequent urination. Seek prompt medical
    review.")
        elif glu >= 126:
            tips.append("**Blood glucose:** if this reading was taken while fasting,
    126 mg/dL or higher is in the "
                        "diabetes range; 100-125 mg/dL is prediabetes. Confirm with
    a repeat test.")
    if not bmi_unknown and p["bmi"] >= 25:
        tips.append(f"**Weight:** BMI {p['bmi']:.1f} is above the healthy range.
```

```
    Losing 5-7% of body weight "
                    "substantially lowers the risk of developing type 2 diabetes.")
    if p["hypertension"] == 1:
        tips.append("**Blood pressure:** hypertension and diabetes often occur
    together - keep blood pressure controlled "
                    "and ask for a glucose test at your next review.")
    if p["heart_disease"] == 1:
        tips.append("**Heart disease:** diabetes raises cardiovascular risk further,
    so a glucose / HbA1c check is "
                    "worthwhile if you haven't had one recently.")
    if p["smoking_history"] == "current":
        tips.append("**Smoking:** smoking raises diabetes and heart risk. Quitting
    is one of the most effective changes you can make.")
    if p["age"] >= 35:
        tips.append("**Age:** adults 35 and over should be screened for diabetes at
    least every 3 years "
                    "(earlier and more often with overweight, high blood pressure or
    family history).")
    if p["gender"] == "Female":
        tips.append("**For women:** tell your doctor if you have had gestational
    diabetes or polycystic ovary syndrome - "
                    "both raise long-term risk.")
    if not has_labs:
        tips.append("**No blood test yet:** this estimate uses only lifestyle and
    history information. "
                    "A blood test (HbA1c or fasting glucose) is the only way to
    know.")
    if bmi_unknown:
        tips.append("**Data quality:** BMI was not provided, so a typical value was
    assumed and the estimate is less reliable.")
    return tips


def clinical_advice(high: bool, tips: list[str], has_labs: bool) -> None:
    st.markdown("#### Recommended next steps")
    if high:
        st.markdown(
            "1. **Arrange a blood test soon** (HbA1c, fasting plasma glucose, or an
    oral glucose tolerance test). "
            "Diagnostic cut-offs: HbA1c ≥ 6.5%, fasting glucose ≥ 126 mg/dL, or
    2-hour glucose ≥ 200 mg/dL.\n"
            "2. **Start protective habits now:** about 150 minutes of moderate
    activity per week, fewer sugary drinks "
            "and refined carbohydrates, more fibre, vegetables and whole grains.\n"
            "3. **Know the warning signs** - excessive thirst, frequent urination,
    unexplained weight loss, blurred vision, "
            "constant tiredness, slow-healing sores - and seek care promptly if they
    appear."
        )
    else:
        st.markdown(
            "1. **Keep it up:** about 150 minutes of moderate activity per week and
    a balanced, high-fibre diet.\n"
            "2. **Stay on a screening schedule:** repeat glucose screening at least
    every 3 years, sooner if weight, "
            "blood pressure or family history change.\n"
            "3. **Know the warning signs** - excessive thirst, frequent urination,
    unexplained weight loss, blurred vision - "
            "and see a clinician if they appear, whatever this result says."
        )
```

```
    if tips:
        st.markdown("#### Based on your inputs")
        for t in tips:
            st.markdown(f"- {t}")


# ---------------------------------------------------------------------------- #
# Sidebar - patient inputs
# ---------------------------------------------------------------------------- #
st.sidebar.header("Patient health parameters")
st.sidebar.caption("■ typical · ■ caution · ■ high-risk range (shown under each
    input)")

patient: dict = {}
patient["gender"] = st.sidebar.radio("Sex", ["Female", "Male"], horizontal=True,
    key="gender",
                                     help="The model was trained on both adult women
    and men.")

patient["age"] = float(st.sidebar.slider("Age (years)", 18, 80, 45, 1, key="age",
                                         help="Trained on adults aged 18-80."))
st.sidebar.caption(f"{LEVEL_ICON[classify('age', patient['age'])[0]]}
    {classify('age', patient['age'])[1]}")

bmi_unknown = st.sidebar.checkbox("BMI: not known", key="na_bmi",
                                  help="Tick if unavailable; a typical value will be
    assumed.")
if bmi_unknown:
    patient["bmi"] = BMI_PLACEHOLDER  # the preprocessor treats this value as
    missing -> median
else:
    patient["bmi"] = float(st.sidebar.slider(
        "BMI (kg/m²)", 12.0, 60.0, 28.0, 0.1, key="bmi",
        help="Body-mass index. Healthy 18.5-24.9; overweight 25-29.9; obese 30 and
    above."))
    lvl, label = classify("bmi", patient["bmi"])
    st.sidebar.caption(f"{LEVEL_ICON[lvl]} {label}")

patient["hypertension"] = int(st.sidebar.checkbox("Diagnosed hypertension (high
    blood pressure)", key="hyp"))
patient["heart_disease"] = int(st.sidebar.checkbox("Diagnosed heart disease",
    key="heart"))
patient["smoking_history"] = st.sidebar.selectbox(
    "Smoking history", ["never", "former", "current", "unknown"], key="smoke",
    format_func=lambda s: {"never": "Never smoked", "former": "Former smoker",
                           "current": "Current smoker", "unknown": "Not known"}[s])

st.sidebar.divider()
has_labs = st.sidebar.toggle("I have recent blood test results", key="has_labs",
                             help="Adds HbA1c and blood glucose, which makes the
    estimate far more accurate.")
if has_labs:
    patient["HbA1c_level"] = float(st.sidebar.slider(
        "HbA1c (%)", 3.5, 9.0, 5.7, 0.1, key="a1c",
        help="Average blood sugar over ~3 months. Normal < 5.7%; prediabetes
    5.7-6.4%; diabetes 6.5% and above."))
    lvl, label = classify("HbA1c_level", patient["HbA1c_level"])
    st.sidebar.caption(f"{LEVEL_ICON[lvl]} {label}")
    patient["blood_glucose_level"] = float(st.sidebar.slider(
        "Blood glucose (mg/dL)", 80, 300, 110, 1, key="glucose",
```

```
        help="Fasting: normal < 100, prediabetes 100-125, diabetes 126 and above.
    Random reading: 200 and above suggests diabetes."))
    lvl, label = classify("blood_glucose_level", patient["blood_glucose_level"])
    st.sidebar.caption(f"{LEVEL_ICON[lvl]} {label}")
else:
    patient["HbA1c_level"], patient["blood_glucose_level"] = np.nan, np.nan  #
    ignored by the screening model

# ------------------------------------------------------------------------ #
# Main panel
# ------------------------------------------------------------------------ #
fs = "full" if has_labs else "screening"
bundle = bundles[fs]
meta, threshold = bundle["meta"], float(bundle["meta"]["decision_threshold"])

st.title("■ Early Diabetes Risk Screening")
st.warning(
    "**Educational screening aid - not a diagnosis.** Trained on a public dataset of
    adult women and men "
    "whose origin is not documented, so it may not generalise to your population. "
    "Only a clinician's laboratory tests can diagnose diabetes."
)
st.caption(f"Model in use: **{meta['model_name']}** · "
           + ("with HbA1c + blood glucose" if has_labs else "screening model (no
    blood test)"))

prob, cleaned, scaled = predict(bundle, patient)
high_risk = prob >= threshold

col_badge, col_gauge = st.columns(2, gap="large")
with col_badge:
    risk_badge(high_risk)
    st.metric("Estimated probability", f"{prob:.1%}")
    tuned = meta["test_metrics_tuned"]
    st.caption(
        f"Screening threshold: **{threshold:.1%}**. At or above it you are flagged
    *High Risk*. The threshold is "
        f"deliberately low - tuned to catch about {meta['recall_target']:.0%} of
    people who have diabetes - so "
        f"about **{1 - tuned['precision']:.0%} of flagged people turn out not to
    have diabetes**. "
        "A High Risk flag means **'get a blood test'**, not 'you have diabetes'."
    )
with col_gauge:
    st.plotly_chart(probability_gauge(prob, high_risk, threshold), width="stretch")

st.divider()
col_advice, col_why = st.columns(2, gap="large")
with col_advice:
    clinical_advice(high_risk, personalised_tips(patient, has_labs, bmi_unknown),
    has_labs)
with col_why:
    st.markdown("#### Why this result? Main contributing factors")
    contributions, unit = explain(bundle, scaled)
    st.plotly_chart(drivers_chart(contributions, unit), width="stretch")
    st.caption("How much each factor raises (red) or lowers (blue) this person's
    risk compared with an average "
               "patient. Derived features are grouped with the measurement they come
    from.")
with st.expander("Values used by the model (after cleaning)"):
```

```
    st.dataframe(cleaned.iloc[0].round(3).rename("value").to_frame(),
    width="stretch")
    st.caption("BMI left blank (or exactly 27.32, the dataset's placeholder value)
    is replaced by the training median.")

with st.expander("About the model"):
    by_sex = meta["test_metrics_by_sex_tuned"]
    lo, hi = meta["test_recall_ci95"]
    st.markdown(
        f"- **Held-out test ROC-AUC:** {meta['test_roc_auc']:.3f} "
        f"({meta['n_test']:,} test patients, {meta['n_train']:,} training
    patients)\n"
        f"- **At the {threshold:.1%} threshold:** recall {tuned['recall']:.0%} (95%
    CI {lo:.0%}-{hi:.0%}), "
        f"precision {tuned['precision']:.0%}, specificity
    {tuned['specificity']:.0%}\n"
        f"- **Women:** recall {by_sex['Female']['recall']:.0%}, ROC-AUC
    {by_sex['Female']['roc_auc']:.3f} · "
        f"**Men:** recall {by_sex['Male']['recall']:.0%}, ROC-AUC
    {by_sex['Male']['roc_auc']:.3f}\n"
        f"- **Trained:** {meta['trained_at_utc']}"
    )
    if not has_labs:
        st.markdown(
            "The no-blood-test model is a genuine *early* screen but a modest one:
    it ranks patients "
            "well above chance (ROC-AUC ≈ 0.80), yet most flagged people are false
    alarms because age, weight "
            "and blood pressure alone cannot separate diabetic from non-diabetic
    people very sharply."
        )
    else:
        st.markdown(
            "With HbA1c and glucose the model is very accurate, but that is because
    these two tests are how "
            "diabetes is *defined* - it is closer to confirming a diagnosis than to
    predicting one early."
        )
```

client/index.html

```
<div id="root"></div><script type="module" src="/src/main.jsx"></script>
```

client/package.json

```
{
  "name": "diabetes-risk-client",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "vite build",
    "preview": "vite preview --host 0.0.0.0"
  },
  "dependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "diabetes-risk-dashboard": "file:..",
    "lucide-react": "^0.468.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "vite": "^6.0.7"
  }
}
```

client/src/main.jsx

```
import React, { useEffect, useMemo, useState } from "react";
import { createRoot } from "react-dom/client";
import { Activity, ArrowRight, Bell, CheckCircle2, ChevronRight, CircleHelp,
    ClipboardCheck, HeartPulse, Info, LogOut, Menu, ShieldCheck, Sparkles,
    UserRound, X } from "lucide-react";
import "./styles.css";

const initialForm = { gender: "Female", age: 45, bmi: 28, bmiUnknown: false,
    hypertension: false, heartDisease: false, smokingHistory: "never", hasLabs:
    false, hba1c: 5.7, bloodGlucose: 110 };

async function readJsonResponse(response) {
  const body = await response.text();
  if (!body.trim()) {
    throw new Error(`The server returned an empty response (HTTP
    ${response.status}). Please make sure the API is running.`);
  }
  let data;
  try {
    data = JSON.parse(body);
  } catch {
    throw new Error(`The server returned an invalid response (HTTP
    ${response.status}).`);
  }
  if (!response.ok) {
    throw new Error(data.message || `Request failed (HTTP ${response.status}).`);
  }
  return data;
}

function App() {
  const [user, setUser] = useState(() =>
    JSON.parse(localStorage.getItem("diabetesUser") || "null"));
  const [login, setLogin] = useState({ email: "", password: "" });
  const [loginError, setLoginError] = useState("");
  const [form, setForm] = useState(initialForm);
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [active, setActive] = useState("Overview");
  const [mobileNav, setMobileNav] = useState(false);

  const update = (key, value) => setForm((current) => ({ ...current, [key]: value
    }));
  const submitLogin = async (event) => {
    event.preventDefault(); setLoginError("");
    try {
      const response = await fetch("/api/auth/login", { method: "POST", headers: {
    "Content-Type": "application/json" }, body: JSON.stringify(login) });
      const data = await readJsonResponse(response);
      localStorage.setItem("diabetesUser", JSON.stringify(data.user));
    setUser(data.user);
    } catch (error) { setLoginError(error.message); }
  };
  const assess = async (event) => {
    event.preventDefault(); setLoading(true);
    try {
      const response = await fetch("/api/predict", { method: "POST", headers: {
    "Content-Type": "application/json" }, body: JSON.stringify({ ...form,
```

```
    heartDisease: form.heartDisease, featureSet: form.hasLabs ? "full" : "screening"
    }) });
      const data = await readJsonResponse(response);
      setResult(data); setActive("Assessment");
    } catch (error) { window.alert(error.message); } finally { setLoading(false); }
  };
  if (!user) return <Login login={login} setLogin={setLogin}
    submitLogin={submitLogin} error={loginError} />;
  return <div className="app-shell">
    <aside className={`sidebar ${mobileNav ? "open" : ""}`}>
      <div className="brand"><div className="brand-mark"><HeartPulse size={22}
    /></div><div><strong>Pulsewise</strong><small>Health
    intelligence</small></div><button className="icon-btn close-nav" onClick={() =>
    setMobileNav(false)}><X size={20} /></button></div>
      <div className="nav-label">Workspace</div>
      {["Overview", "Assessment", "History"].map((item) => <button key={item}
    className={`nav-item ${active === item ? "active" : ""}`} onClick={() => {
    setActive(item); setMobileNav(false); }}><Activity size={18}
    />{item}<ChevronRight size={15} /></button>)}
      <div className="nav-label settings-label">Support</div>
      <button className="nav-item"><CircleHelp size={18} />How it works</button>
      <div className="sidebar-bottom"><div className="privacy"><ShieldCheck
    size={18} /><div><strong>Your privacy matters</strong><span>Data stays in this
    session.</span></div></div><button className="nav-item logout" onClick={() => {
    localStorage.removeItem("diabetesUser"); setUser(null); }}><LogOut size={18}
    />Sign out</button></div>
    </aside>
    <main className="main-content">
      <header className="topbar"><button className="icon-btn menu-btn" onClick={()
    => setMobileNav(true)}><Menu size={22} /></button><div
    className="crumb"><span>Workspace</span><ChevronRight size={14}
    /><strong>{active}</strong></div><div className="top-actions"><button
    className="icon-btn notification"><Bell size={19} /><i /></button><div
    className="avatar">{user.name[0].toUpperCase()}</div><span
    className="user-name">{user.name}</span></div></header>
      <section className="page">
        <div className="welcome"><div><p className="eyebrow">SATURDAY, OCTOBER 31,
    2026</p><h1>{active === "Overview" ? <>Good morning, <em>{user.name.split("
    ")[0]}</em></> : active}</h1><p className="subhead">A clearer view of your
    metabolic health starts here.</p></div><button className="primary-btn"
    onClick={() => setActive("Assessment")}><Sparkles size={17} />Start
    assessment</button></div>
        {active === "Overview" && <Overview result={result} setActive={setActive}
    />}
        {active === "Assessment" && <Assessment form={form} update={update}
    assess={assess} loading={loading} result={result} />}
        {active === "History" && <History result={result} setActive={setActive} />}
      </section>
    </main>
  </div>;
}

function Login({ login, setLogin, submitLogin, error }) {
  return <div className="login-page"><div className="login-art"><div
    className="art-glow" /><div className="login-brand"><div
    className="brand-mark"><HeartPulse size={22}
    /></div><strong>Pulsewise</strong></div><div className="art-copy"><p
    className="eyebrow">PROACTIVE HEALTH, MADE SIMPLE</p><h1>Small signals.<br
    /><em>Better decisions.</em></h1><p>Understand your diabetes risk with a
    thoughtful, science-led screening experience.</p></div><div
```

```
    className="testimonial"><div className="quote">“The clarity I needed to take the
    next step.”</div><small>— Pulsewise member</small></div></div><div
    className="login-panel"><div className="mobile-login-brand"><div
    className="brand-mark"><HeartPulse size={22}
    /></div><strong>Pulsewise</strong></div><div className="login-form"><p
    className="eyebrow">WELCOME BACK</p><h2>Your health, at a glance.</h2><p
    className="form-copy">Sign in to continue to your personal health
    dashboard.</p><form onSubmit={submitLogin}><label>Email address<input
    type="email" placeholder="you@example.com" value={login.email} onChange={(e) =>
    setLogin({ ...login, email: e.target.value })} required
    /></label><label>Password<div className="password-wrap"><input type="password"
    placeholder="Enter your password" value={login.password} onChange={(e) =>
    setLogin({ ...login, password: e.target.value })} required minLength="6"
    /><span>Forgot?</span></div></label>{error && <div
    className="error">{error}</div>}<button className="primary-btn full"
    type="submit">Sign in <ArrowRight size={17} /></button></form><div
    className="demo-hint">Demo mode · any valid email and 6+ character
    password</div></div></div></div>;
}

function Overview({ result, setActive }) {
  return <><div className="stat-grid"><div className="stat-card accent"><div
    className="stat-icon"><Activity size={20} /></div><div
    className="stat-label">Latest assessment</div><strong>{result ?
    `${(result.probability * 100).toFixed(1)}%` : "Not yet
    taken"}</strong><span>{result ? (result.highRisk ? "Follow-up recommended" :
    "Within lower-risk range") : "Start your first check-in"}</span></div><div
    className="stat-card"><div className="stat-icon lavender"><ClipboardCheck
    size={20} /></div><div className="stat-label">Assessment
    status</div><strong>{result ? "Complete" : "Ready when you
    are"}</strong><span>{result ? "Updated just now" : "Takes about 2
    minutes"}</span></div><div className="stat-card"><div className="stat-icon
    peach"><ShieldCheck size={20} /></div><div className="stat-label">Your
    data</div><strong>Private</strong><span>Stored only for this
    session</span></div></div><div className="dashboard-grid"><div
    className="welcome-card"><div className="card-kicker">YOUR NEXT BEST
    STEP</div><h2>A healthier future starts with knowing your baseline.</h2><p>Our
    assessment uses your everyday health information to estimate risk and help you
    decide whether a clinical blood test could be useful.</p><button
    className="text-btn" onClick={() => setActive("Assessment")}>Take assessment
    <ArrowRight size={16} /></button><div className="line-art"><div /><div /><div
    /></div></div><div className="insight-card"><div
    className="card-heading"><h3>How Pulsewise works</h3><Info size={17}
    /></div><Step n="01" title="Share your context" text="A few details about your
    health and lifestyle." /><Step n="02" title="Get a clear signal" text="A model
    trained on 79,000+ adult records." /><Step n="03" title="Choose your next step"
    text="Use the result to have a better conversation." /></div></div><div
    className="disclaimer"><Info size={16} /><span><strong>Educational screening
    only.</strong> Pulsewise is not a diagnosis or a substitute for professional
    medical advice.</span></div></>;
}
function Step({ n, title, text }) { return <div className="step"><span>{n}</span><di
    v><strong>{title}</strong><p>{text}</p></div><ChevronRight size={15} /></div>; }

function Assessment({ form, update, assess, loading, result }) {
  return <div className="assessment-layout"><form className="assessment-form"
    onSubmit={assess}><div className="section-title"><div><p
    className="eyebrow">STEP 1 OF 1</p><h2>Tell us about yourself</h2><p>Use your
    most recent information. It does not need to be perfect.</p></div><span
    className="secure"><ShieldCheck size={15} />Secure session</span></div><div
```

```
    className="input-section"><h3>About you</h3><div className="segmented"><span
    className="field-label">Sex</span><div>{["Female", "Male"].map((x) => <button
    type="button" className={form.gender === x ? "selected" : ""} onClick={() =>
    update("gender", x)} key={x}>{x}</button>)}</div></div><div
    className="two-col"><Field label="Age" suffix="years"><input type="number"
    min="18" max="80" value={form.age} onChange={(e) => update("age",
    e.target.value)} /></Field><Field label="BMI" suffix="kg/m²"><input
    type="number" min="12" max="60" step=".1" value={form.bmi}
    disabled={form.bmiUnknown} onChange={(e) => update("bmi", e.target.value)}
    /></Field></div><label className="check-row"><input type="checkbox"
    checked={form.bmiUnknown} onChange={(e) => update("bmiUnknown",
    e.target.checked)} /><span>BMI is not known</span><small>We'll use a typical
    value</small></label></div><div className="input-section"><h3>Health
    context</h3><Toggle label="Diagnosed hypertension" checked={form.hypertension}
    onChange={(v) => update("hypertension", v)} /><Toggle label="Diagnosed heart
    disease" checked={form.heartDisease} onChange={(v) => update("heartDisease", v)}
    /><div className="select-row"><label>Smoking history<select
    value={form.smokingHistory} onChange={(e) => update("smokingHistory",
    e.target.value)}><option value="never">Never smoked</option><option
    value="former">Former smoker</option><option value="current">Current
    smoker</option><option value="unknown">Not
    known</option></select></label></div></div><div className="input-section
    lab-section"><div className="lab-heading"><div><h3>Recent blood test
    results</h3><p>Have your HbA1c and glucose values handy?</p></div><button
    type="button" className={`switch ${form.hasLabs ? "on" : ""}`} onClick={() =>
    update("hasLabs", !form.hasLabs)}><i /></button></div>{form.hasLabs && <div
    className="two-col"><Field label="HbA1c" suffix="%"><input type="number"
    min="3.5" max="9" step=".1" value={form.hba1c} onChange={(e) => update("hba1c",
    e.target.value)} /></Field><Field label="Blood glucose" suffix="mg/dL"><input
    type="number" min="80" max="300" value={form.bloodGlucose} onChange={(e) =>
    update("bloodGlucose", e.target.value)} /></Field></div>}</div><button
    className="primary-btn submit-btn" disabled={loading}>{loading ?
    "Calculating..." : "Calculate my risk"} <ArrowRight size={17}
    /></button></form><ResultCard result={result} /></div>;
}
function Field({ label, suffix, children }) { return <label className="field"><span>
    {label}</span><div>{children}<small>{suffix}</small></div></label>; }
function Toggle({ label, checked, onChange }) { return <div
    className="toggle-row"><span>{label}</span><button type="button"
    className={`switch ${checked ? "on" : ""}`} onClick={() =>
    onChange(!checked)}><i /></button></div>; }
function ResultCard({ result }) { if (!result) return <div
    className="result-empty"><div className="empty-icon"><Sparkles size={24}
    /></div><h3>Your result will appear here</h3><p>Complete the assessment to see
    your estimated risk, contributing factors, and a suggested next step.</p></div>;
    return <div className="result-panel"><div className="result-top"><p
    className="eyebrow">YOUR RESULT</p><span className={`result-pill
    ${result.highRisk ? "high" : "low"}`}>{result.highRisk ? "Needs follow-up" :
    "Lower risk"}</span></div><h2>{(result.probability *
    100).toFixed(1)}<small>%</small></h2><p className="result-description">Estimated
    probability based on your {result.featureSet === "full" ? "health and lab" :
    "health"} information.</p><div className="meter"><div style={{ width:
    `${Math.min(result.probability * 100 * 1.8, 100)}%` }} /></div><div
    className="meter-labels"><span>Lower</span><span>Threshold {(result.threshold *
    100).toFixed(1)}%</span><span>Higher</span></div><div
    className="result-note"><Info size={16} /><span>{result.highRisk ? "This is a
    screening flag, not a diagnosis. Consider arranging a blood test and speaking
    with a clinician." : "Keep up healthy habits and stay on a routine screening
    schedule."}</span></div><h3 className="drivers-title">Main contributing
    factors</h3>{result.drivers.map((driver) => <div className="driver"
```

```
    key={driver.name}><span>{driver.name}</span><div className="driver-bar"><i
    className={driver.value > 0 ? "positive" : ""} style={{ width:
    `${Math.min(Math.abs(driver.value) * 100, 100)}%` }} /></div><b
    className={driver.value > 0 ? "positive-text" : ""}>{driver.value > 0 ? "+" :
    ""}{driver.value.toFixed(2)}</b></div>)}</div>; }
function History({ result, setActive }) { return <div className="history-card"><div
    className="card-heading"><div><p className="eyebrow">PRIVATE SESSION
    HISTORY</p><h2>Your assessments</h2></div><button className="primary-btn"
    onClick={() => setActive("Assessment")}>New assessment</button></div>{result ?
    <div className="history-row"><div
    className="history-date"><strong>Today</strong><span>{result.featureSet ===
    "full" ? "Full assessment" : "Screening assessment"}</span></div><div
    className={`result-pill ${result.highRisk ? "high" : "low"}`}>{result.highRisk ?
    "Needs follow-up" : "Lower risk"}</div><strong>{(result.probability *
    100).toFixed(1)}%</strong><ChevronRight size={17} /></div> : <div
    className="empty-history"><ClipboardCheck size={28} /><h3>No assessments
    yet</h3><p>Your completed assessments will appear here.</p></div>}</div>; }

createRoot(document.getElementById("root")).render(<App />);
```

client/src/styles.css

```
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&f
    amily=Manrope:wght@500;600;700;800&display=swap');
:root{font-family:"DM Sans",sans-serif;color:#14251f;background:#f7f9f8;font-synthes
    is:none}*{box-sizing:border-box}body{margin:0}.app-shell{display:flex;min-height
    :100vh}.sidebar{width:256px;background:#102e27;color:#dce9e4;padding:25px 16px;d
    isplay:flex;flex-direction:column;flex-shrink:0}.brand,.login-brand,.mobile-logi
    n-brand{display:flex;align-items:center;gap:11px;color:#fff}.brand-mark{backgrou
    nd:#b4e6d3;color:#0c4d3d;border-radius:11px;width:38px;height:38px;display:grid;
    place-items:center}.brand strong,.login-brand strong,.mobile-login-brand
    strong{font-family:Manrope;font-size:18px;letter-spacing:-.4px}.brand small{disp
    lay:block;color:#89aaa0;font-size:10px;margin-top:2px}.nav-label{font-size:10px;
    text-transform:uppercase;letter-spacing:1.6px;color:#74978c;font-weight:700;marg
    in:56px 13px 12px}.nav-label.settings-label{margin-top:42px}.nav-item{border:0;b
    ackground:transparent;color:#a8c1b9;width:100%;padding:12px
    13px;border-radius:10px;display:flex;align-items:center;gap:12px;font:500 14px
    "DM Sans";cursor:pointer;text-align:left}.nav-item svg:last-child{margin-left:au
    to;opacity:0}.nav-item:hover,.nav-item.active{color:#fff;background:#1f493e}.nav
    -item.active
    svg:last-child{opacity:1}.sidebar-bottom{margin-top:auto}.privacy{border-top:1px
    solid #2a5147;border-bottom:1px solid #2a5147;padding:18px
    4px;display:flex;gap:10px;color:#a5c4ba}.privacy strong,.privacy
    span{display:block;font-size:11px}.privacy strong{color:#dce9e4;margin-bottom:3p
    x}.logout{margin-top:13px}.main-content{flex:1;min-width:0}.topbar{height:75px;b
    order-bottom:1px solid #e6ece9;display:flex;align-items:center;padding:0 5.2%;ga
    p:10px;background:#fff}.crumb{display:flex;align-items:center;gap:8px;font-size:
    13px;color:#97a9a3}.crumb strong{color:#29423a;font-weight:600}.top-actions{marg
    in-left:auto;display:flex;align-items:center;gap:13px}.icon-btn{border:0;backgro
    und:transparent;color:#628075;cursor:pointer;position:relative;padding:6px;displ
    ay:flex}.notification i{width:6px;height:6px;background:#e17665;border:1px solid
    #fff;position:absolute;border-radius:50%;right:5px;top:3px}.avatar{height:34px;w
    idth:34px;border-radius:50%;display:grid;place-items:center;background:#d7eee5;c
    olor:#226451;font-weight:700}.user-name{font-size:13px;text-transform:capitalize
    ;color:#314c43}.menu-btn,.close-nav{display:none}.page{max-width:1220px;margin:a
    uto;padding:43px 5.2% 60px}.welcome{display:flex;justify-content:space-between;a
    lign-items:end;margin-bottom:37px}.eyebrow{font-size:10px;letter-spacing:1.5px;c
    olor:#78948a;font-weight:700;margin:0 0 11px}.welcome h1{font:800 31px
    Manrope;margin:0;color:#153a30;letter-spacing:-1px}.welcome h1 em{font-style:nor
    mal;color:#299372}.subhead{color:#80958e;font-size:14px;margin:9px 0 0}.primary-
    btn{border:0;background:#187a61;color:#fff;border-radius:8px;padding:12px
    17px;font:600 13px "DM
    Sans";display:flex;align-items:center;gap:9px;cursor:pointer;box-shadow:0 3px
    8px #187a6128}.primary-btn:hover{background:#11634f}.stat-grid{display:grid;grid
    -template-columns:repeat(3,1fr);gap:17px}.stat-card,.welcome-card,.insight-card,
    .history-card,.assessment-form,.result-panel,.result-empty{background:#fff;borde
    r:1px solid #e6eeea;border-radius:13px;box-shadow:0 5px 20px
    #173a2d08}.stat-card{padding:21px 22px;position:relative}.stat-icon{width:37px;h
    eight:37px;border-radius:9px;background:#dff3ea;color:#218363;display:grid;place
    -items:center;margin-bottom:20px}.stat-icon.lavender{background:#e8e7fb;color:#6
    964b6}.stat-icon.peach{background:#fce8dd;color:#cf7b53}.stat-label{font-size:11
    px;color:#82948e;margin-bottom:7px}.stat-card strong{display:block;font:700 20px
    Manrope;color:#1b3d33}.stat-card span{font-size:12px;color:#8da19a;margin-top:5p
    x;display:block}.dashboard-grid{display:grid;grid-template-columns:1.15fr
    .85fr;gap:17px;margin-top:17px}.welcome-card{background:#edf7f2;padding:27px 29p
    x;position:relative;overflow:hidden;min-height:263px}.card-kicker{font-size:10px
    ;color:#398b72;font-weight:700;letter-spacing:1.5px}.welcome-card h2{font:700
    22px Manrope;line-height:1.35;max-width:410px;letter-spacing:-.5px;margin:12px 0
    9px;color:#1e4d3e}.welcome-card p{color:#658178;line-height:1.55;font-size:13px;
    max-width:445px;margin:0}.text-btn{border:0;background:transparent;color:#19785e
```

```
;font:700 13px "DM Sans";padding:17px 0;display:flex;align-items:center;gap:8px;
cursor:pointer}.line-art{position:absolute;right:20px;bottom:-17px;opacity:.5}.l
ine-art div{border:1px solid #8cc9b3;width:150px;height:80px;border-radius:50%;t
ransform:rotate(-30deg);margin-bottom:-49px}.insight-card{padding:25px 26px}.car
d-heading{display:flex;align-items:center;justify-content:space-between}.card-he
ading h3,.insight-card h3{font:700 15px
Manrope;color:#1c4035;margin:0}.card-heading svg{color:#8da8a0}.step{display:fle
x;align-items:center;gap:13px;margin-top:21px}.step>span{font:700 10px
Manrope;color:#58a58b;background:#e5f4ed;border-radius:6px;padding:7px
6px}.step>div{flex:1}.step strong{font-size:12px}.step
p{font-size:11px;color:#93a49e;margin:3px 0 0}.step>svg{color:#b4c6bf}.disclaime
r{display:flex;gap:9px;align-items:center;margin:26px
4px;color:#8b9b95;font-size:11px}.disclaimer svg{color:#79a698}.disclaimer
strong{color:#57756a}.assessment-layout{display:grid;grid-template-columns:1.1fr
.9fr;gap:19px;align-items:start}.assessment-form{padding:28px 30px}.section-titl
e{display:flex;justify-content:space-between;align-items:start;border-bottom:1px
solid #edf1ef;padding-bottom:22px}.section-title h2,.history-card h2{font:700
21px Manrope;color:#193e32;margin:0 0 6px}.section-title p:not(.eyebrow){font-si
ze:12px;color:#8a9b95;margin:0}.secure{display:flex;align-items:center;gap:5px;c
olor:#59927e;font-size:10px}.input-section{border-bottom:1px solid
#edf1ef;padding:23px 0}.input-section h3{font:700 14px
Manrope;color:#27463c;margin:0 0 18px}.segmented{display:flex;align-items:center
;justify-content:space-between;margin-bottom:20px}.field-label,.field>span{font-
size:12px;color:#6d827a;font-weight:600}.segmented>div{display:flex;background:#
f2f5f3;border-radius:7px;padding:3px}.segmented
button{border:0;background:transparent;color:#80948c;padding:8px
26px;border-radius:5px;font:600 12px "DM Sans";cursor:pointer}.segmented
button.selected{background:#fff;color:#267b61;box-shadow:0 2px 6px
#173a2d12}.two-col{display:grid;grid-template-columns:1fr
1fr;gap:13px}.field>div{display:flex;align-items:center;border:1px solid
#dfe9e4;border-radius:7px;margin-top:7px;padding:0 11px;background:#fff}.field
input,.select-row select{border:0;outline:0;width:100%;padding:10px
0;background:transparent;color:#294a3e;font:500 13px "DM Sans"}.field small{colo
r:#94a59f;font-size:11px}.check-row{display:flex;align-items:center;gap:8px;marg
in-top:14px;color:#657c73;font-size:11px}.check-row
input{accent-color:#218665}.check-row small{color:#a1afa9}.toggle-row{display:fl
ex;justify-content:space-between;align-items:center;padding:13px
0;border-top:1px solid #f0f3f1;color:#587168;font-size:12px}.switch{width:34px;h
eight:19px;border:0;border-radius:20px;background:#dce5e1;padding:2px;cursor:poi
nter}.switch i{display:block;width:15px;height:15px;border-radius:50%;background
:#fff;transition:.2s}.switch.on{background:#329a78}.switch.on
i{transform:translateX(15px)}.select-row label{display:block;color:#6d827a;font-
size:12px;font-weight:600;margin-top:10px}.select-row
select{display:block;border:1px solid #dfe9e4;border-radius:7px;padding:10px;mar
gin-top:7px}.lab-heading{display:flex;align-items:center;justify-content:space-b
etween}.lab-heading h3{margin:0 0 5px}.lab-heading p{font-size:11px;color:#8b9e9
6;margin:0}.submit-btn{margin-top:22px;width:100%;justify-content:center}.submit
-btn:disabled{opacity:.6;cursor:wait}.result-panel,.result-empty{padding:28px
27px;position:sticky;top:20px}.result-empty{text-align:center;padding:80px 38px}
.empty-icon{background:#e6f4ee;color:#329577;width:51px;height:51px;display:grid
;place-items:center;border-radius:50%;margin:0 auto 17px}.result-empty
h3{font:700 16px Manrope;color:#34564b;margin:0 0 9px}.result-empty p{font-size:
12px;color:#93a39d;line-height:1.55}.result-top{display:flex;justify-content:spa
ce-between;align-items:start}.result-panel h2{font:800 46px
Manrope;color:#1b4f3e;letter-spacing:-2px;margin:18px 0 3px}.result-panel h2 sma
ll{font-size:24px;letter-spacing:0}.result-description{font-size:12px;color:#849
790;margin:0 0
20px}.result-pill{font-size:10px;font-weight:700;border-radius:20px;padding:6px
10px}.result-pill.high{color:#ac5144;background:#fde9e5}.result-pill.low{color:#
227554;background:#e2f5eb}.meter{height:8px;background:#e7eeea;border-radius:8px
```

```
;overflow:hidden}.meter div{height:100%;border-radius:8px;background:linear-grad
ient(90deg,#61bd96,#e2a56b)}.meter-labels{display:flex;justify-content:space-bet
ween;color:#9baba5;font-size:9px;margin-top:7px}.result-note{display:flex;gap:8p
x;background:#f3f8f5;border-radius:8px;padding:12px;margin:22px
0;color:#658077;font-size:11px;line-height:1.45}.result-note
svg{flex-shrink:0;color:#51957d}.drivers-title{font:700 13px
Manrope;color:#315247;border-top:1px solid #edf1ef;padding-top:20px;margin:0 0 1
5px}.driver{display:flex;align-items:center;gap:8px;font-size:10px;color:#70867d
;margin:12px 0}.driver>span{width:105px}.driver-bar{height:6px;background:#edf2e
f;flex:1;border-radius:5px}.driver-bar
i{display:block;height:100%;background:#e4a494;border-radius:5px}.driver-bar
i:not(.positive){background:#8ac6ae}.driver b{font-size:10px;color:#7e9f91;width
:35px;text-align:right}.positive-text{color:#cc7567!important}.history-card{padd
ing:29px}.history-card .card-heading{margin-bottom:28px}.history-row{border:1px
solid #e8efeb;border-radius:9px;padding:16px
18px;display:flex;align-items:center;gap:20px}.history-date{flex:1}.history-date
strong,.history-date span{display:block}.history-date
strong{font-size:13px;color:#315449}.history-date span{font-size:11px;color:#94a
39e;margin-top:4px}.empty-history{text-align:center;padding:65px;color:#97a7a1}.
empty-history svg{color:#8bbbaa}.empty-history h3{font:700 16px
Manrope;color:#466257}.empty-history p{font-size:12px}.login-page{min-height:100
vh;display:grid;grid-template-columns:1.05fr .95fr;background:#f8faf9}.login-art
{background:#123a30;position:relative;overflow:hidden;color:#f1faf6;padding:45px
7%;display:flex;flex-direction:column}.login-art:after{content:"";position:absol
ute;width:570px;height:570px;border:1px solid
#68ad96;opacity:.18;border-radius:50%;right:-270px;bottom:-180px;box-shadow:0 0
0 70px #123a30,0 0 0 71px #68ad9633,0 0 0 140px #123a3022,0 0 0 141px #68ad9622}
.art-glow{position:absolute;width:400px;height:400px;border-radius:50%;backgroun
d:#2b947244;filter:blur(20px);top:20%;left:-170px}.art-copy{position:relative;z-
index:1;margin:auto 0}.art-copy .eyebrow{color:#8ed1b7}.art-copy h1{font:800
clamp(38px,5vw,63px) Manrope;line-height:1.08;letter-spacing:-2px;margin:14px 0
20px}.art-copy h1 em{font-style:normal;color:#8ed1b7}.art-copy>p:last-child{max-
width:330px;line-height:1.7;color:#a7c8bb;font-size:15px}.testimonial{position:r
elative;z-index:1;margin-top:auto}.quote{font:500 16px
Manrope;margin-bottom:9px}.testimonial small{color:#8fb4a6;font-size:11px}.login
-panel{display:flex;align-items:center;justify-content:center;padding:40px}.logi
n-form{width:100%;max-width:370px}.login-form h2{font:800 30px Manrope;letter-sp
acing:-1px;color:#193e32;margin:0}.form-copy{font-size:13px;color:#8a9c95;margin
:10px 0 34px}.login-form
label{display:block;color:#506c61;font-size:12px;font-weight:600;margin:18px
0}.login-form input{display:block;width:100%;border:1px solid
#dce8e2;border-radius:7px;padding:13px
14px;margin-top:7px;outline:0;color:#294a3e;font:13px "DM
Sans";background:#fff}.login-form input:focus{border-color:#50a986;box-shadow:0
0 0 3px #50a98618}.password-wrap{position:relative}.password-wrap
input{padding-right:60px}.password-wrap span{position:absolute;right:12px;bottom
:13px;color:#278266;font-size:10px}.full{width:100%;justify-content:center;margi
n-top:28px;padding:13px}.demo-hint{text-align:center;font-size:10px;color:#9aaba
4;margin-top:18px}.mobile-login-brand{display:none}.error{background:#fde9e5;col
or:#a94e44;padding:10px;border-radius:7px;font-size:11px}.close-nav{color:#c5ddd
4}
@media(max-width:900px){.sidebar{position:fixed;z-index:5;height:100vh;left:-270px;t
ransition:.2s}.sidebar.open{left:0}.menu-btn,.close-nav{display:flex}.close-nav{
margin-left:auto}.topbar{padding:0 25px}.page{padding:32px 25px}.user-name{displ
ay:none}.dashboard-grid,.assessment-layout{grid-template-columns:1fr}.result-pan
el{position:static}.result-empty{padding:50px 38px}.login-page{grid-template-col
umns:1fr}.login-art{display:none}.mobile-login-brand{display:flex;color:#193e32;
margin-bottom:55px}.login-panel{padding:45px 8%}.stat-grid{grid-template-columns
:1fr}.welcome{align-items:start;gap:20px;flex-direction:column}.welcome
h1{font-size:27px}}
```

```
@media(max-width:520px){.page{padding:26px 15px}.topbar{padding:0
    15px;height:65px}.welcome{margin-bottom:25px}.welcome .primary-btn{width:100%;ju
    stify-content:center}.assessment-form,.result-panel,.result-empty,.history-card{
    padding:21px 18px}.segmented button{padding:8px 17px}.two-col{grid-template-colu
    mns:1fr}.history-row{gap:10px;padding:12px}.history-row
    .result-pill{display:none}}
```

client/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: { port: 5173, proxy: { "/api": "http://127.0.0.1:3001" } }
});
```

export.py

```
from pathlib import Path
import textwrap
from xml.sax.saxutils import escape

from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Preformatted, Spacer, PageBreak
)

ROOT = Path(__file__).parent
OUTPUT = ROOT / "Project_Source_Code.pdf"

EXTENSIONS = {
    ".py", ".js", ".jsx", ".ts", ".tsx",
    ".html", ".css", ".scss", ".json",
    ".java", ".cpp", ".c", ".h", ".hpp",
    ".sql", ".xml", ".yaml", ".yml",
    ".md", ".sh", ".bat"
}

EXCLUDED_DIRS = {
    "node_modules", ".git", ".venv", "venv",
    "__pycache__", ".pytest_cache", ".mypy_cache",
    ".ruff_cache", "dist", "build", ".next",
    "coverage", ".idea", ".vscode"
}

EXCLUDED_FILES = {
    "package-lock.json", "yarn.lock", "pnpm-lock.yaml",
    "export_code_pdf.py", "Project_Source_Code.pdf"
}

styles = getSampleStyleSheet()
title_style = styles["Title"]

file_style = ParagraphStyle(
    "FileName",
    parent=styles["Heading2"],
    fontSize=12,
    leading=16,
    spaceBefore=10,
    spaceAfter=8,
    keepWithNext=1,
)

code_style = ParagraphStyle(
    "Code",
    fontName="Courier",
    fontSize=9,
    leading=12,
    leftIndent=5,
    rightIndent=5,
)

def collect_files():
    files = []
```

```
    for path in ROOT.rglob("*"):
        if not path.is_file():
            continue

        relative = path.relative_to(ROOT)

        if any(part in EXCLUDED_DIRS for part in relative.parts):
            continue

        if path.name in EXCLUDED_FILES:
            continue

        if path.suffix.lower() not in EXTENSIONS:
            continue

        files.append(path)

    return sorted(files)

def wrap_code(code, width=84):
    result = []

    for line in code.splitlines():
        if len(line) <= width:
            result.append(line)
        else:
            result.extend(
                textwrap.wrap(
                    line,
                    width=width,
                    subsequent_indent="    ",
                    break_long_words=True,
                    break_on_hyphens=False,
                )
            )

    return "\n".join(result)

def build_pdf():
    doc = SimpleDocTemplate(
        str(OUTPUT),
        pagesize=A4,
        rightMargin=18 * mm,
        leftMargin=18 * mm,
        topMargin=18 * mm,
        bottomMargin=18 * mm,
    )

    story = [
        Paragraph("Project Source Code", title_style),
        Spacer(1, 8),
    ]

    files = collect_files()

    story.append(Paragraph(
        f"Total source files: {len(files)}",
        styles["Normal"]
    ))
    story.append(PageBreak())
```

```
    for index, file in enumerate(files):
        if index:
            story.append(PageBreak())

        relative = file.relative_to(ROOT).as_posix()

        try:
            code = file.read_text(
                encoding="utf-8",
                errors="replace"
            )
        except Exception:
            continue

        story.append(Paragraph(escape(relative), file_style))
        story.append(Preformatted(
            wrap_code(code),
            code_style,
            maxLineLength=84
        ))

    doc.build(story)
    print(f"PDF created: {OUTPUT}")

if __name__ == "__main__":
    build_pdf()
```

models/full/model_metadata.json


```
{
  "feature_set": "full",
  "feature_set_description": "with HbA1c + glucose",
  "model_name": "XGBoost",
  "decision_threshold": 0.16141246259212494,
  "default_threshold": 0.5,
  "recall_target": 0.85,
  "raw_columns": [
    "gender",
    "age",
    "hypertension",
    "heart_disease",
    "smoking_history",
    "bmi",
    "HbA1c_level",
    "blood_glucose_level"
  ],
  "model_features": [
    "is_male",
    "age",
    "hypertension",
    "heart_disease",
    "smoking_never",
    "smoking_former",
    "smoking_current",
    "smoking_unknown",
    "bmi",
    "Age_Group",
    "BMI_Category",
    "Age_BMI_Risk",
    "Cardiometabolic_Burden",
    "HbA1c_level",
    "blood_glucose_level",
    "HbA1c_Category",
    "Glucose_Category",
    "Glucose_eAG_Gap"
  ],
  "cv_roc_auc_mean": 0.9751976861277634,
  "test_roc_auc": 0.9747705176799413,
  "test_pr_auc": 0.8876682142216621,
  "test_metrics_default": {
    "threshold": 0.5,
    "accuracy": 0.9651960475800868,
    "precision": 0.9795744680851064,
    "recall": 0.6851190476190476,
    "specificity": 0.9983109296924485,
    "f1": 0.8063047285464098,
    "tn": 14185,
    "fp": 24,
    "fn": 529,
    "tp": 1151
  },
  "test_metrics_tuned": {
    "threshold": 0.16141246259212494,
    "accuracy": 0.9313990811253068,
    "precision": 0.6301853486319505,
    "recall": 0.85,
    "specificity": 0.9410232950946583,
```

```
    "f1": 0.7237709072478459,
    "tn": 13371,
    "fp": 838,
    "fn": 252,
    "tp": 1428
  },
  "test_recall_ci95": [
    0.8321611823759063,
    0.8667058398327793
  ],
  "test_precision_ci95": [
    0.6087098681722942,
    0.6487674692247184
  ],
  "test_metrics_by_sex_tuned": {
    "Female": {
      "threshold": 0.16141246259212494,
      "accuracy": 0.9434160706773244,
      "precision": 0.6526891522333638,
      "recall": 0.8201603665521191,
      "specificity": 0.9558772437753329,
      "f1": 0.7269035532994924,
      "tn": 8254,
      "fp": 381,
      "fn": 157,
      "tp": 716,
      "roc_auc": 0.9751687204967131,
      "n": 9508,
      "n_diabetic": 873
    },
    "Male": {
      "threshold": 0.16141246259212494,
      "accuracy": 0.9134931828866949,
      "precision": 0.6090675791274593,
      "recall": 0.8822800495662949,
      "specificity": 0.9180121994976678,
      "f1": 0.7206477732793523,
      "tn": 5117,
      "fp": 457,
      "fn": 95,
      "tp": 712,
      "roc_auc": 0.9733881283654995,
      "n": 6381,
      "n_diabetic": 807
    }
  },
  "n_train": 63555,
  "n_test": 15889,
  "positive_rate": 0.1057348572579427,
  "cleaning_log": {
    "rows_raw": 100000,
    "after_removing_other_gender": 99982,
    "after_keeping_adults": 82767,
    "after_removing_duplicates": 79444
  },
  "trained_at_utc": "2026-10-31T10:13:45+00:00",
  "sklearn_version": "1.9.0",
  "random_state": 42
}
```

models/screening/model_metadata.json


```
{
  "feature_set": "screening",
  "feature_set_description": "screening (no lab values)",
  "model_name": "Random Forest",
  "decision_threshold": 0.07743928499010524,
  "default_threshold": 0.5,
  "recall_target": 0.85,
  "raw_columns": [
    "gender",
    "age",
    "hypertension",
    "heart_disease",
    "smoking_history",
    "bmi"
  ],
  "model_features": [
    "is_male",
    "age",
    "hypertension",
    "heart_disease",
    "smoking_never",
    "smoking_former",
    "smoking_current",
    "smoking_unknown",
    "bmi",
    "Age_Group",
    "BMI_Category",
    "Age_BMI_Risk",
    "Cardiometabolic_Burden"
  ],
  "cv_roc_auc_mean": 0.8028390649939047,
  "test_roc_auc": 0.7981043411452835,
  "test_pr_auc": 0.324917774142279,
  "test_metrics_default": {
    "threshold": 0.5,
    "accuracy": 0.895776952608723,
    "precision": 0.6304347826086957,
    "recall": 0.034523809523809526,
    "specificity": 0.9976071503976353,
    "f1": 0.0654627539503386,
    "tn": 14175,
    "fp": 34,
    "fn": 1622,
    "tp": 58
  },
  "test_metrics_tuned": {
    "threshold": 0.07743928499010524,
    "accuracy": 0.6147019950909434,
    "precision": 0.19583675705286224,
    "recall": 0.8511904761904762,
    "specificity": 0.5867407980857203,
    "f1": 0.3184146069917613,
    "tn": 8337,
    "fp": 5872,
    "fn": 250,
    "tp": 1430
  },
  "test_recall_ci95": [
```

```
    0.8350572003675955,
    0.8688918100816884
  ],
  "test_precision_ci95": [
    0.18689816502211698,
    0.20400547800251903
  ],
  "test_metrics_by_sex_tuned": {
    "Female": {
      "threshold": 0.07743928499010524,
      "accuracy": 0.6474547749263778,
      "precision": 0.18404282436910527,
      "recall": 0.827033218785796,
      "specificity": 0.6292993630573248,
      "f1": 0.3010842368640534,
      "tn": 5434,
      "fp": 3201,
      "fn": 151,
      "tp": 722,
      "roc_auc": 0.8064247438599005,
      "n": 9508,
      "n_diabetic": 873
    },
    "Male": {
      "threshold": 0.07743928499010524,
      "accuracy": 0.5658987619495377,
      "precision": 0.20952944658182895,
      "recall": 0.8773234200743495,
      "specificity": 0.52081090778615,
      "f1": 0.33827042522694695,
      "tn": 2903,
      "fp": 2671,
      "fn": 99,
      "tp": 708,
      "roc_auc": 0.7810865324890879,
      "n": 6381,
      "n_diabetic": 807
    }
  },
  "n_train": 63555,
  "n_test": 15889,
  "positive_rate": 0.1057348572579427,
  "cleaning_log": {
    "rows_raw": 100000,
    "after_removing_other_gender": 99982,
    "after_keeping_adults": 82767,
    "after_removing_duplicates": 79444
  },
  "trained_at_utc": "2026-10-31T10:14:42+00:00",
  "sklearn_version": "1.9.0",
  "random_state": 42
}
```

package.json

```
{
  "name": "diabetes-risk-dashboard",
  "private": true,
  "scripts": {
    "install:all": "npm install --prefix client && npm install --prefix server",
    "dev": "concurrently \"npm run dev --prefix server\" \"npm run dev --prefix
    client\"",
    "start": "npm run start --prefix server"
  },
  "devDependencies": {
    "concurrently": "^9.1.2"
  }
}
```

predict.py

```
import json
import sys
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

from preprocessing import BMI_PLACEHOLDER, FEATURE_GROUPS, FEATURE_SETS,
    ClinicalPreprocessor

ROOT = Path(__file__).parent


def positive_class_shap(values):
    if isinstance(values, list):
        values = values[1]
    values = np.asarray(values)
    return values[:, :, 1] if values.ndim == 3 else values


def load_bundle(feature_set):
    model_dir = ROOT / "models" / feature_set
    model = joblib.load(model_dir / "best_model.joblib")
    metadata = json.loads((model_dir / "model_metadata.json").read_text())
    return {
        "model": model,
        "prep": ClinicalPreprocessor.from_params_dict(
            joblib.load(model_dir / "imputation_params.joblib")
        ),
        "scaler": joblib.load(model_dir / "scaler.joblib"),
        "meta": metadata,
    }


def main():
    request = json.load(sys.stdin)
    feature_set = request["featureSet"]
    bundle = load_bundle(feature_set)
    patient = {
        "gender": request["gender"],
        "age": float(request["age"]),
        "hypertension": int(bool(request["hypertension"])),
        "heart_disease": int(bool(request["heartDisease"])),
        "smoking_history": request["smokingHistory"],
        "bmi": BMI_PLACEHOLDER if request.get("bmiUnknown") else
    float(request["bmi"]),
    }
    if feature_set == "full":
        patient["HbA1c_level"] = float(request["hba1c"])
        patient["blood_glucose_level"] = float(request["bloodGlucose"])

    cleaned = bundle["prep"].transform(pd.DataFrame([patient]))
    scaled = pd.DataFrame(
        np.asarray(bundle["scaler"].transform(cleaned)), columns=cleaned.columns
    )
    probability = float(bundle["model"].predict_proba(scaled)[0, 1])
    model = bundle["model"]
```

```
    if hasattr(model, "coef_"):
        contributions = pd.Series(
            model.coef_[0] * scaled.iloc[0].to_numpy(), index=scaled.columns
        )
        unit = "log-odds"
    else:
        import shap
        contributions = pd.Series(
            positive_class_shap(shap.TreeExplainer(model).shap_values(scaled))[0],
            index=scaled.columns,
        )
        unit = "log-odds" if type(model).__name__.startswith("XGB") else
    "probability"
    grouped = contributions.groupby(lambda feature: FEATURE_GROUPS.get(feature,
    feature))
    drivers = [
        {"name": name, "value": round(float(value), 6)}
        for name, value in grouped.sum().sort_values(key=lambda values:
    values.abs(), ascending=False).head(6).items()
    ]
    threshold = float(bundle["meta"]["decision_threshold"])
    print(json.dumps({
        "probability": probability,
        "threshold": threshold,
        "highRisk": probability >= threshold,
        "modelName": bundle["meta"]["model_name"],
        "featureSet": feature_set,
        "drivers": drivers,
        "driverUnit": unit,
        "cleaned": {key: round(float(value), 4) for key, value in
    cleaned.iloc[0].items()},
        "metrics": {
            "rocAuc": bundle["meta"]["test_roc_auc"],
            "recall": bundle["meta"]["test_metrics_tuned"]["recall"],
            "precision": bundle["meta"]["test_metrics_tuned"]["precision"],
            "specificity": bundle["meta"]["test_metrics_tuned"]["specificity"],
        },
    }))


if __name__ == "__main__":
    main()
```

preprocessing.py

```
"""
preprocessing.py
================
Shared, leak-free clinical preprocessing for the both-sexes Diabetes Prediction
    project.

The same code is used by ``train.py`` (inside every cross-validation fold) and by
``app.py`` (at inference time), so training and serving can never drift apart.

Two feature sets are supported
------------------------------
* ``"full"``      - demographics + history + BMI + HbA1c + blood glucose.
* ``"screening"`` - the same WITHOUT the two lab values. In this dataset the label
    is
  almost fully determined by HbA1c / glucose, so the "full" model is closer to
  diagnosis than to early prediction. The "screening" model answers the genuinely
  early question: "who should be tested?" using only information a person has
  before any blood test.

Steps performed by :class:`ClinicalPreprocessor`
------------------------------------------------
1. Encode sex (Male = 1) and smoking history (never / former / current / unknown).
2. Treat the BMI placeholder value (27.32 - see note below) as missing data.
3. Median imputation using medians learned from the TRAINING data only.
4. IQR clipping with fences learned from the TRAINING data only (observed values).
5. Medically motivated feature engineering (see ``add_engineered_features``).

Note on the BMI placeholder: 24% of adult rows have BMI exactly 27.32, the median of
the original data. It is an upstream imputation marker, not a measurement, so it is
treated as missing here. No "was-missing" indicator is added on purpose: those rows
have a different diabetes rate (7.9% vs 11.4%) for reasons unrelated to biology, and
    an
indicator would teach the model a data-collection artefact.
"""
from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

# ---------------------------------------------------------------------------- #
# Dataset constants
# ---------------------------------------------------------------------------- #
TARGET = "diabetes"
ID_COLUMNS = ["EmployeeName", "Patient_id"]  # dropped if present (not clinical
    data)

RAW_BASE = ["gender", "age", "hypertension", "heart_disease", "smoking_history",
    "bmi"]
RAW_LABS = ["HbA1c_level", "blood_glucose_level"]
FEATURE_SETS = ("full", "screening")
RAW_COLUMNS = {"full": RAW_BASE + RAW_LABS, "screening": RAW_BASE}
ALL_RAW_COLUMNS = RAW_BASE + RAW_LABS

SEX_VALUES = ["Female", "Male"]  # 'Other' (18 rows, all non-diabetic) is excluded
MIN_AGE = 18  # type-2 diabetes prediction is meaningless for infants / children
```

```
# Raw smoking labels -> 4 clinically coherent groups
SMOKING_MAP = {
    "never": "never",
    "former": "former",
    "not current": "former",
    "ever": "former",
    "current": "current",
    "No Info": "unknown",
}
SMOKING_CATEGORIES = ["never", "former", "current", "unknown"]

BMI_PLACEHOLDER = 27.32  # upstream median-imputation marker -> treated as missing

# Tukey's "far-out" fence (3 x IQR): trims only extreme / implausible values and
    keeps
# the clinically meaningful tails (e.g. severe obesity, very high HbA1c).
IQR_FACTOR = 3.0
CLIP_COLUMNS = ["age", "bmi", "HbA1c_level", "blood_glucose_level"]

# Bin edges (np.digitize: a value equal to an edge falls in the UPPER bin)
AGE_BIN_EDGES = [35, 45, 55, 65]  # <35, 35-44, 45-54, 55-64, 65+   (screening
    starts at 35)
BMI_BIN_EDGES = [18.5, 25.0, 30.0]  # underweight, healthy, overweight, obese (WHO)
HBA1C_BIN_EDGES = [5.7, 6.5]  # normal, prediabetes, diabetes range (ADA)
GLUCOSE_BIN_EDGES = [100.0, 140.0, 200.0]  # <100, 100-139, 140-199, >=200 mg/dL

# Features that are summarised under one clinical heading in explanations.
FEATURE_GROUPS = {
    "age": "Age", "Age_Group": "Age", "Age_BMI_Risk": "Age",
    "bmi": "BMI", "BMI_Category": "BMI",
    "is_male": "Sex",
    "hypertension": "Hypertension", "heart_disease": "Heart disease",
    "Cardiometabolic_Burden": "Hypertension / heart disease",
    "smoking_never": "Smoking", "smoking_former": "Smoking",
    "smoking_current": "Smoking", "smoking_unknown": "Smoking",
    "HbA1c_level": "HbA1c", "HbA1c_Category": "HbA1c", "Glucose_eAG_Gap": "HbA1c",
    "blood_glucose_level": "Blood glucose", "Glucose_Category": "Blood glucose",
}


def model_features(feature_set: str) -> list[str]:
    """Ordered list of columns the model sees for a feature set."""
    feats = ["is_male", "age", "hypertension", "heart_disease",
             *[f"smoking_{c}" for c in SMOKING_CATEGORIES], "bmi",
             "Age_Group", "BMI_Category", "Age_BMI_Risk", "Cardiometabolic_Burden"]
    if feature_set == "full":
        feats += ["HbA1c_level", "blood_glucose_level",
                  "HbA1c_Category", "Glucose_Category", "Glucose_eAG_Gap"]
    return feats


# ---------------------------------------------------------------------------- #
# Loading and dataset-level cleaning
# ---------------------------------------------------------------------------- #
def load_data(path: str | Path) -> tuple[pd.DataFrame, dict]:
    """Load the CSV and apply dataset-level cleaning. Returns (frame, cleaning_log).

    * drops any name / patient-id columns (not clinical features)
    * keeps Male / Female rows only (the 18 'Other' rows are all non-diabetic and
    far
```

```
      too few to learn from)
    * keeps adults (age >= 18)
    * removes exact duplicate rows BEFORE the train/test split, so a patient cannot
      appear in both sets
    """
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(
            f"Dataset not found at '{path}'. Place diabetes_prediction_dataset.csv
    there."
        )
    df = pd.read_csv(path).drop(columns=ID_COLUMNS, errors="ignore")
    missing = [c for c in ALL_RAW_COLUMNS + [TARGET] if c not in df.columns]
    if missing:
        raise ValueError(f"Dataset is missing columns: {missing}")

    log = {"rows_raw": len(df)}
    df = df[df["gender"].isin(SEX_VALUES)]
    log["after_removing_other_gender"] = len(df)
    df = df[df["age"] >= MIN_AGE]
    log["after_keeping_adults"] = len(df)
    df = df.drop_duplicates().reset_index(drop=True)
    log["after_removing_duplicates"] = len(df)
    return df, log


# ---------------------------------------------------------------------------- #
# Feature engineering
# ---------------------------------------------------------------------------- #
def add_engineered_features(df: pd.DataFrame, feature_set: str) -> pd.DataFrame:
    """Add clinically motivated features to an already cleaned frame.

    * ``Age_Group``              ordinal age bucket (<35, 35-44, 45-54, 55-64, 65+)
    * ``BMI_Category``           WHO class: 0 underweight, 1 healthy, 2 overweight,
    3 obese
    * ``Age_BMI_Risk``           age x BMI / 1000 - risk compounds when both are
    high
    * ``Cardiometabolic_Burden`` hypertension + heart disease (0-2)
    * ``HbA1c_Category``         0 normal (<5.7), 1 prediabetes (5.7-6.4), 2
    diabetes range (>=6.5)
    * ``Glucose_Category``       0: <100, 1: 100-139, 2: 140-199, 3: >=200 mg/dL
    * ``Glucose_eAG_Gap``        blood glucose minus the glucose predicted from
    HbA1c
      (eAG = 28.7 x HbA1c - 46.7, ADAG formula). Large gaps flag a mismatch between
      long-term and current glycaemia.
    """
    out = df.copy()
    out["Age_Group"] = np.digitize(out["age"].to_numpy(),
    AGE_BIN_EDGES).astype(float)
    out["BMI_Category"] = np.digitize(out["bmi"].to_numpy(),
    BMI_BIN_EDGES).astype(float)
    out["Age_BMI_Risk"] = out["age"] * out["bmi"] / 1000.0
    out["Cardiometabolic_Burden"] = out["hypertension"] + out["heart_disease"]
    if feature_set == "full":
        out["HbA1c_Category"] = np.digitize(out["HbA1c_level"].to_numpy(),
    HBA1C_BIN_EDGES).astype(float)
        out["Glucose_Category"] = np.digitize(out["blood_glucose_level"].to_numpy(),
    GLUCOSE_BIN_EDGES).astype(float)
        out["Glucose_eAG_Gap"] = out["blood_glucose_level"] - (28.7 *
    out["HbA1c_level"] - 46.7)
```

```
    return out


class ClinicalPreprocessor(BaseEstimator, TransformerMixin):
    """Encoding, placeholder->NaN, median imputation, IQR clipping, feature
    engineering.

    Everything that depends on data (medians, clip fences) is learned in ``fit`` so
    that, inside cross-validation or a train/test split, nothing leaks from the
    validation / test rows.
    """

    def __init__(self, feature_set: str = "full", iqr_factor: float = IQR_FACTOR):
        self.feature_set = feature_set
        self.iqr_factor = iqr_factor

    # -- helpers ------------------------------------------------------------ #
    def _prepare(self, X) -> pd.DataFrame:
        """Select the raw columns for this feature set and encode them
    numerically."""
        if self.feature_set not in FEATURE_SETS:
            raise ValueError(f"feature_set must be one of {FEATURE_SETS}")
        cols = RAW_COLUMNS[self.feature_set]
        raw = pd.DataFrame(X).loc[:, cols].copy()

        sex = raw["gender"].map({"Female": 0.0, "Male": 1.0})
        if sex.isna().any():
            bad = sorted(raw.loc[sex.isna(), "gender"].astype(str).unique())
            raise ValueError(f"Unsupported gender value(s): {bad}. Use 'Female' or
    'Male'.")

        df = pd.DataFrame({"is_male": sex})
        for c in ["age", "hypertension", "heart_disease", "bmi"]:
            df[c] = pd.to_numeric(raw[c], errors="coerce").astype(float)
        df.loc[df["bmi"].round(2) == BMI_PLACEHOLDER, "bmi"] = np.nan  # upstream
    placeholder
        for c in RAW_LABS:
            if c in raw.columns:
                df[c] = pd.to_numeric(raw[c], errors="coerce").astype(float)
        # Unrecognised smoking labels are treated as "unknown" rather than failing.
        df["smoking_history"] =
    raw["smoking_history"].map(SMOKING_MAP).fillna("unknown")
        return df

    # -- sklearn API -------------------------------------------------------- #
    def fit(self, X, y=None):
        df = self._prepare(X)
        numeric = [c for c in df.columns if c != "smoking_history"]
        self.medians_ = df[numeric].median()
        clip_cols = [c for c in CLIP_COLUMNS if c in df.columns]
        q1, q3 = df[clip_cols].quantile(0.25), df[clip_cols].quantile(0.75)
        iqr = q3 - q1
        self.lower_ = q1 - self.iqr_factor * iqr
        self.upper_ = q3 + self.iqr_factor * iqr
        self.model_features_ = model_features(self.feature_set)
        return self

    def transform(self, X) -> pd.DataFrame:
        df = self._prepare(X)
        smoking = df.pop("smoking_history")
```

```
    df = df.fillna(self.medians_)
    for c in self.lower_.index:
        df[c] = df[c].clip(lower=self.lower_[c], upper=self.upper_[c])
    for cat in SMOKING_CATEGORIES:
        df[f"smoking_{cat}"] = (smoking == cat).astype(float)
    return add_engineered_features(df, self.feature_set)[self.model_features_]

def get_feature_names_out(self, input_features=None):
    return np.array(model_features(self.feature_set), dtype=object)

def count_clipped(self, X) -> pd.Series:
    """Number of values per clipped column outside the learned IQR fences."""
    df = self._prepare(X).drop(columns="smoking_history").fillna(self.medians_)
    return pd.Series({c: int(((df[c] < self.lower_[c]) | (df[c] >
self.upper_[c])).sum())
                      for c in self.lower_.index})

# -- (de)serialisation of the learned parameters ------------------------- #
def get_params_dict(self) -> dict:
    """Plain-python dict of everything learned in ``fit`` (safe to
joblib/JSON)."""
    return {
        "feature_set": self.feature_set,
        "iqr_factor": float(self.iqr_factor),
        "raw_columns": list(RAW_COLUMNS[self.feature_set]),
        "model_features": list(self.model_features_),
        "medians": {k: float(v) for k, v in self.medians_.items()},
        "clip_lower": {k: float(v) for k, v in self.lower_.items()},
        "clip_upper": {k: float(v) for k, v in self.upper_.items()},
        "bmi_placeholder": BMI_PLACEHOLDER,
        "smoking_map": dict(SMOKING_MAP),
    }

@classmethod
def from_params_dict(cls, params: dict) -> "ClinicalPreprocessor":
    """Rebuild a fitted preprocessor from ``get_params_dict`` output."""
    obj = cls(feature_set=params["feature_set"],
iqr_factor=params["iqr_factor"])
    obj.medians_ = pd.Series(params["medians"])
    obj.lower_ = pd.Series(params["clip_lower"])
    obj.upper_ = pd.Series(params["clip_upper"])
    obj.model_features_ = list(params["model_features"])
    return obj
```

README.md

````
# Early Diabetes Prediction - adult women AND men

Both-sexes version of the diabetes project. It replaces the Pima dataset (women
    only) with the
100,000-row "Diabetes prediction dataset" (female and male, with HbA1c and blood
    glucose).

```bash
pip install -r requirements.txt
python train.py --data data/diabetes_prediction_dataset.csv   # ~2-3 min, writes
    ./models and ./outputs
streamlit run app.py
```

## React + Node dashboard

The project also includes a React/Vite dashboard and an Express API. The Node API
    keeps
the trained Python preprocessing and model artifacts as the source of truth by
    invoking
`predict.py` for each assessment.

```bash
npm install
npm run install:all
npm run dev
```

Open `http://127.0.0.1:5173`. The API runs on `http://127.0.0.1:3001`. Login is a
local demo flow: use any email address and a password with at least six characters.
This authentication is not intended for production use.

## Two models, because the labs change everything

| | Inputs | Test ROC-AUC | Recall | Precision | Use |
|---|---|---|---|---|---|
| **screening** | sex, age, BMI, hypertension, heart disease, smoking | ~0.80 | 85%
    | ~20% | "Who should get a blood test?" - a real early screen |
| **full** | screening inputs + HbA1c + blood glucose | ~0.975 | 85% | ~63% | Close
    to confirming a diagnosis |

In this dataset the label follows HbA1c / glucose thresholds almost exactly (e.g.
    every row with
HbA1c above 6.6 is positive), so a model that sees the labs is closer to
    *re-applying the diagnostic
definition* than to *predicting early*. The screening model is the honest
    early-prediction model; the
app picks one automatically (toggle "I have recent blood test results").

## What changed from the Pima version

| Topic | Pima version | Both-sexes version |
|---|---|---|
| Population | 768 women (Pima heritage) | ~79k adults, 60% women / 40% men |
| Biological zeros | zeros in 5 columns -> NaN | none; instead the **BMI placeholder
    27.32** (24% of rows, an upstream median-fill marker) -> NaN |
| Sex | n/a | `is_male` feature; tested separately per sex |
| Smoking | n/a | 6 raw labels -> never / former / current / unknown |
````

| Features | Pregnancies, SkinThickness, Insulin, pedigree... | hypertension, heart disease, smoking, HbA1c, glucose |
| Engineered | BMI/glucose ratio, HOMA proxy | Age_Group, BMI_Category, Age_BMI_Risk, Cardiometabolic_Burden, HbA1c/Glucose categories, glucose-vs-HbA1c gap |

Dataset cleaning (printed by `train.py`)

100,000 rows -> drop 18 `Other` gender rows (all non-diabetic, too few to learn from) -> keep adults 18+ (17% of rows were children; type-2 prediction is not meaningful for them) -> remove 3,300+ exact duplicates **before** the split so no patient is in both train and test -> **79,444 rows, 10.6% diabetic**.
The name / patient-ID columns in the source file are dropped and not saved.

Design decisions

* **No leakage:** the split comes first; medians, IQR fences and scaler statistics are learned from the training fold only (the whole chain is one sklearn `Pipeline`, refit in every CV fold).
* **Threshold tuned on out-of-fold training predictions**, never the test set. It is the highest-precision threshold with recall >= 85%. Test recall at that threshold is ~85% (95% CI ~83-87%).
* **Random Forest without `class_weight`:** reweighting inflates predicted probabilities, and the app shows probabilities to users. Recall is reached through the threshold instead.
* **Same threshold for women and men**; `train.py` prints recall / precision / ROC-AUC per sex. Recall is a little lower for women (~0.83 vs ~0.88) - worth knowing if you act on it.
* **No "BMI was missing" indicator:** rows with the placeholder BMI have a different diabetes rate for data-collection reasons, not biology.
* **Explanations** group derived features under the measurement they come from (e.g. Age_Group -> Age).

Limitations - please read

* **Unknown provenance.** The dataset's origin, collection method and label definition are not documented. The labs-to-label relationship above is unusually clean, which is a warning sign that the data may be simulated or rule-labelled. I could not verify either way. Do not assume it reflects a real population.
* **Not validated on any external data**, and no demographic information (ethnicity, country) is available.
* **Screening model precision is ~20%**: about 4 in 5 flagged people do not have diabetes. That is the cost of 85% recall with weak predictors, and a flag only ever means "get a blood test".
* Blood glucose in the data has no stated context (fasting vs random), so the app's glucose guidance presents both.
* Educational tool, **not a medical device**.

server/index.js

```
import cors from "cors";
import express from "express";
import { spawn } from "node:child_process";
import { fileURLToPath } from "node:url";
import path from "node:path";

const app = express();
const port = Number(process.env.PORT || 3001);
const root = path.dirname(fileURLToPath(import.meta.url));
const projectRoot = path.resolve(root, "..");
const python = process.env.PYTHON || "C:/Users/Saurabh
    Kumar/AppData/Local/Programs/Python/Python312/python.exe";

app.use(cors());
app.use(express.json());

app.get("/api/health", (_req, res) => res.json({ status: "ok", service:
    "diabetes-risk-api" }));

app.post("/api/auth/login", (req, res) => {
  const { email, password } = req.body ?? {};
  if (!email || !password) {
    return res.status(400).json({ message: "Email and password are required." });
  }
  if (password.length < 6) {
    return res.status(401).json({ message: "Use a password with at least 6
    characters." });
  }
  return res.json({
    user: { name: email.split("@")[0].replace(/[._-]/g, " "), email },
    token: `demo-${Buffer.from(email).toString("base64url")}`
  });
});

app.post("/api/predict", (req, res) => {
  const input = req.body;
  if (!input || !["screening", "full"].includes(input.featureSet)) {
    return res.status(400).json({ message: "featureSet must be screening or full."
    });
  }

  const child = spawn(python, [path.join(projectRoot, "predict.py")], {
    cwd: projectRoot,
    stdio: ["pipe", "pipe", "pipe"]
  });
  let output = "";
  let errors = "";
  child.stdout.on("data", (chunk) => { output += chunk.toString(); });
  child.stderr.on("data", (chunk) => { errors += chunk.toString(); });
  child.on("error", (error) => res.status(500).json({ message: `Python worker
    failed: ${error.message}` }));
  child.on("close", (code) => {
    if (code !== 0) {
      return res.status(500).json({ message: "Prediction failed.", detail:
    errors.slice(-1000) });
    }
    try {
      return res.json(JSON.parse(output));
```

```
    } catch {
      return res.status(500).json({ message: "Prediction returned an invalid
    response.", detail: errors.slice(-1000) });
    }
  });
  child.stdin.end(JSON.stringify(input));
});

app.listen(port, () => {
  console.log(`Diabetes Risk API listening on http://localhost:${port}`);
});
```

server/package.json

```
{
  "name": "diabetes-risk-api",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "node --watch index.js",
    "start": "node index.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "diabetes-risk-dashboard": "file:..",
    "express": "^4.21.2"
  }
}
```

train.py

```
#!/usr/bin/env python3
"""
train.py - Early Diabetes Prediction for BOTH sexes (adult men and women)

Run:  python train.py --data data/diabetes_prediction_dataset.csv

For each feature set ("full" = with HbA1c / glucose, "screening" = without labs):
1. Stratified 80/20 split (random_state=42; identical rows for both feature sets)
2. 5-fold stratified CV benchmark of Logistic Regression / Random Forest / XGBoost.
   Every fold re-fits the WHOLE pipeline (encode -> impute -> clip -> engineer ->
   StandardScaler -> model), so nothing leaks from validation folds.
3. Best model chosen by mean CV ROC-AUC
4. Decision threshold tuned on out-of-fold training predictions so Recall >= 85%
   (the test set is never used to pick the threshold)
5. Held-out test evaluation at the default (0.50) and the tuned threshold, plus a
   sensitivity breakdown for women and men
6. SHAP explanations
7. Artifacts saved with joblib / JSON under models/<feature_set>/
"""
from __future__ import annotations

import argparse
import json
from datetime import datetime, timezone
from pathlib import Path

import joblib
import matplotlib

matplotlib.use("Agg")  # headless backend: figures are written to disk
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn
from matplotlib.colors import LinearSegmentedColormap
from sklearn.base import clone
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier  # swap for lightgbm.LGBMClassifier if you prefer

from preprocessing import (
    ALL_RAW_COLUMNS,
    FEATURE_GROUPS,
    FEATURE_SETS,
```

```
    TARGET,
    ClinicalPreprocessor,
    load_data,
    model_features,
)

# ---------------------------------------------------------------------------- #
# Configuration
# ---------------------------------------------------------------------------- #
SEED = 42
TEST_SIZE = 0.20
N_FOLDS = 5
DEFAULT_THRESHOLD = 0.50
CLASS_NAMES = ["Non-diabetic", "Diabetic"]
SHAP_SAMPLE = 2000  # rows used for SHAP (keeps tree explainers fast)

MODEL_COLORS = {
    "Logistic Regression": "#2a78d6",  # blue
    "Random Forest": "#eb6834",  # orange
    "XGBoost": "#1baf7a",  # aqua
}
INK, INK_2, SURFACE, GRID = "#0b0b0b", "#52514e", "#fcfcfb", "#e4e3df"
SEQ_CMAP = LinearSegmentedColormap.from_list("seq_blue", ["#cde2fb", "#0d366b"])

plt.rcParams.update(
    {
        "figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor":
    SURFACE,
        "text.color": INK, "axes.labelcolor": INK_2, "axes.edgecolor": GRID,
        "xtick.color": INK_2, "ytick.color": INK_2,
        "axes.titlesize": 12, "axes.titleweight": "bold", "axes.titlelocation":
    "left",
        "axes.spines.top": False, "axes.spines.right": False,
        "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.8,
    "axes.axisbelow": True,
        "legend.frameon": False, "font.size": 10, "lines.linewidth": 2,
    }
)

SET_TITLES = {"full": "with HbA1c + glucose", "screening": "screening (no lab
    values)"}


def section(title: str) -> None:
    print(f"\n{'=' * 78}\n{title}\n{'=' * 78}")


# ---------------------------------------------------------------------------- #
# Models and pipeline
# ---------------------------------------------------------------------------- #
def build_models() -> dict:
    """The three benchmark models (regularised: the label is noisy)."""
    return {
        "Logistic Regression": LogisticRegression(max_iter=3000, random_state=SEED),
        "Random Forest": RandomForestClassifier(
            # No class_weight: it would inflate probabilities, and the app shows
    them to
            # the user. The recall target is met through threshold tuning instead.
            n_estimators=200, max_depth=8, min_samples_leaf=30, max_features="sqrt",
            n_jobs=-1, random_state=SEED,
```

```
        ),
        "XGBoost": XGBClassifier(
            n_estimators=400, learning_rate=0.05, max_depth=4, min_child_weight=5,
            subsample=0.8, colsample_bytree=0.8, reg_lambda=2.0,
            eval_metric="logloss", n_jobs=-1, random_state=SEED,
        ),
    }


def make_pipeline(model, feature_set: str) -> Pipeline:
    """Preprocess -> scale -> classify. Fitting this on a training fold learns the
    medians, IQR fences and scaler statistics from that fold only."""
    return Pipeline(
        [
            ("prep", ClinicalPreprocessor(feature_set=feature_set)),
            ("scaler", StandardScaler().set_output(transform="pandas")),
            ("clf", model),
        ]
    )


# ---------------------------------------------------------------------------- #
# Metrics and threshold helpers
# ---------------------------------------------------------------------------- #
def metrics_at_threshold(y_true, proba, thr: float) -> dict:
    pred = (np.asarray(proba) >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0, 1]).ravel()
    return {
        "threshold": float(thr),
        "accuracy": float((tp + tn) / len(y_true)),
        "precision": float(precision_score(y_true, pred, zero_division=0)),
        "recall": float(recall_score(y_true, pred, zero_division=0)),
        "specificity": float(tn / (tn + fp)) if tn + fp else float("nan"),
        "f1": float(f1_score(y_true, pred, zero_division=0)),
        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
    }


def select_threshold(y_true, proba, min_recall: float) -> float:
    """Highest-precision threshold whose recall is >= min_recall (ties -> higher
    threshold). If no threshold reaches the target, the best-recall one is
    returned."""
    precision, recall, thresholds = precision_recall_curve(y_true, proba)
    precision, recall = precision[:-1], recall[:-1]
    eligible = np.where(recall >= min_recall)[0]
    if eligible.size == 0:
        return float(thresholds[np.argmax(recall)])
    best_precision = precision[eligible].max()
    ties = eligible[np.isclose(precision[eligible], best_precision)]
    return float(thresholds[ties[-1]])


def bootstrap_ci(y_true, proba, thr, n_boot=1000, seed=SEED):
    """95% bootstrap CI for recall and precision at a threshold."""
    rng = np.random.default_rng(seed)
    y_true, proba = np.asarray(y_true), np.asarray(proba)
    recalls, precisions = [], []
    for _ in range(n_boot):
        idx = rng.integers(0, len(y_true), len(y_true))
        yt, pred = y_true[idx], proba[idx] >= thr
```

```
        tp, fn = np.sum(pred & (yt == 1)), np.sum(~pred & (yt == 1))
        fp = np.sum(pred & (yt == 0))
        if tp + fn == 0:
            continue
        recalls.append(tp / (tp + fn))
        precisions.append(tp / (tp + fp) if tp + fp else np.nan)
    return np.nanpercentile(recalls, [2.5, 97.5]), np.nanpercentile(precisions,
    [2.5, 97.5])


def print_threshold_report(title, y_true, proba, thr) -> dict:
    m = metrics_at_threshold(y_true, proba, thr)
    pred = (np.asarray(proba) >= thr).astype(int)
    print(f"\n--- {title} (threshold = {thr:.3f}) ---")
    print("Confusion matrix  [rows = actual, cols = predicted]")
    print(pd.DataFrame(
        [[m["tn"], m["fp"]], [m["fn"], m["tp"]]],
        index=[f"Actual {c}" for c in CLASS_NAMES],
        columns=[f"Pred {c}" for c in CLASS_NAMES],
    ).to_string())
    print(classification_report(y_true, pred, target_names=CLASS_NAMES, digits=3))
    return m


# ---------------------------------------------------------------------------- #
# Cross-validated benchmark (manual loop: one fit per fold gives BOTH the fold
# scores and the out-of-fold probabilities used for threshold tuning)
# ---------------------------------------------------------------------------- #
def cv_benchmark(models: dict, X_train: pd.DataFrame, y_train: np.ndarray,
    feature_set: str):
    cv = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=SEED)
    rows, oof_probas = [], {}
    for name, model in models.items():
        oof = np.zeros(len(y_train))
        folds = []
        for tr, va in cv.split(X_train, y_train):
            pipe = make_pipeline(clone(model), feature_set).fit(X_train.iloc[tr],
    y_train[tr])
            p = pipe.predict_proba(X_train.iloc[va])[:, 1]
            oof[va] = p
            m = metrics_at_threshold(y_train[va], p, DEFAULT_THRESHOLD)
            folds.append({
                "roc_auc": roc_auc_score(y_train[va], p),
                "pr_auc": average_precision_score(y_train[va], p),
                "recall": m["recall"], "precision": m["precision"],
                "f1": m["f1"], "accuracy": m["accuracy"],
            })
        fold_df = pd.DataFrame(folds)
        row = {"model": name}
        for key in fold_df.columns:
            row[f"{key}_mean"], row[f"{key}_std"] = fold_df[key].mean(),
    fold_df[key].std()
        rows.append(row)
        oof_probas[name] = oof
        print(f"  {name:<20} ROC-AUC {row['roc_auc_mean']:.4f} ±
    {row['roc_auc_std']:.4f}")
    return pd.DataFrame(rows), oof_probas


# ---------------------------------------------------------------------------- #
```

```
# Plotting
# ---------------------------------------------------------------------------- #
def plot_model_comparison(cv_df: pd.DataFrame, path: Path, title_suffix: str) ->
    None:
    metrics = [("roc_auc", "ROC-AUC"), ("pr_auc", "PR-AUC"), ("recall", "Recall @
    0.50"),
               ("precision", "Precision @ 0.50")]
    fig, ax = plt.subplots(figsize=(9, 4.6))
    width, x = 0.2, np.arange(len(metrics))
    for i, name in enumerate(cv_df["model"]):
        row = cv_df.iloc[i]
        ax.bar(x + (i - 1) * (width + 0.02), [row[f"{k}_mean"] for k, _ in metrics],
    width,
               yerr=[row[f"{k}_std"] for k, _ in metrics], label=name,
    color=MODEL_COLORS[name],
               error_kw={"ecolor": INK_2, "lw": 1, "capsize": 3})
    ax.set_xticks(x, [lbl for _, lbl in metrics])
    ax.set(ylim=(0, 1.0), ylabel="5-fold CV score (mean ± SD)",
           title=f"Model benchmark - {title_suffix}")
    ax.legend(loc="upper center", ncol=3, bbox_to_anchor=(0.5, -0.1))
    ax.grid(axis="x", visible=False)
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    plt.close(fig)


def plot_roc_pr(y_test, test_probas: dict, best: str, thr_opt: float, path: Path,
                recall_target: float, title_suffix: str) -> None:
    fig, (ax_roc, ax_pr) = plt.subplots(1, 2, figsize=(12, 5))
    for name, proba in test_probas.items():
        fpr, tpr, _ = roc_curve(y_test, proba)
        ax_roc.plot(fpr, tpr, color=MODEL_COLORS[name],
                    label=f"{name} (AUC {roc_auc_score(y_test, proba):.3f})")
        prec, rec, _ = precision_recall_curve(y_test, proba)
        ax_pr.plot(rec, prec, color=MODEL_COLORS[name],
                   label=f"{name} (AP {average_precision_score(y_test,
    proba):.3f})")
    ax_roc.plot([0, 1], [0, 1], ls="--", color=INK_2, lw=1, label="Chance")
    ax_roc.set(xlabel="False positive rate (1 - specificity)", ylabel="True positive
    rate (recall)",
               title=f"ROC curve - test set, {title_suffix}")
    ax_roc.legend(loc="lower right")
    for thr, label, marker in [(DEFAULT_THRESHOLD, "Default 0.50", "o"),
                               (thr_opt, f"Tuned {thr_opt:.2f}", "D")]:
        m = metrics_at_threshold(y_test, test_probas[best], thr)
        ax_pr.plot(m["recall"], m["precision"], marker, ms=9, mfc=INK, mec=SURFACE,
    mew=2,
                   label=f"{best}: {label}", zorder=5)
    ax_pr.axvline(recall_target, color=INK_2, ls=":", lw=1.2)
    ax_pr.text(recall_target + 0.01, 0.05, f"recall target {recall_target:.0%}",
    color=INK_2, fontsize=9)
    ax_pr.axhline(np.mean(y_test), color=INK_2, ls="--", lw=1)
    ax_pr.set(xlabel="Recall (sensitivity)", ylabel="Precision", ylim=(0, 1.02),
              title=f"Precision-Recall - test set, {title_suffix}")
    ax_pr.legend(loc="upper right", fontsize=8.5)
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    plt.close(fig)
def plot_threshold_tuning(y_true, oof_proba, thr_opt, recall_target, path: Path,
    title_suffix) -> None:
```

```
    grid = np.linspace(0.02, 0.95, 187)
    rows = [metrics_at_threshold(y_true, oof_proba, t) for t in grid]
    fig, ax = plt.subplots(figsize=(9, 4.8))
    ax.plot(grid, [r["recall"] for r in rows], color="#2a78d6", label="Recall")
    ax.plot(grid, [r["precision"] for r in rows], color="#eb6834",
    label="Precision")
    ax.plot(grid, [r["f1"] for r in rows], color="#1baf7a", label="F1")
    ax.axhline(recall_target, color=INK_2, ls=":", lw=1.2)
    ax.axvline(DEFAULT_THRESHOLD, color=INK_2, ls="--", lw=1)
    ax.axvline(thr_opt, color=INK, lw=1.5)
    ax.text(DEFAULT_THRESHOLD + 0.01, 0.04, "default 0.50", color=INK_2, fontsize=9)
    ax.text(thr_opt + 0.01, 0.12, f"tuned {thr_opt:.2f}", color=INK, fontsize=9)
    ax.set(xlabel="Decision threshold", ylabel="Score (out-of-fold, training set)",
    ylim=(0, 1.02),
           title=f"Threshold tuning - {title_suffix}")
    ax.legend(loc="upper center", ncol=3, bbox_to_anchor=(0.5, -0.14))
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    plt.close(fig)


def plot_confusions(y_test, proba, thr_default, thr_opt, path: Path, title_suffix)
    -> None:
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.4))
    for ax, thr, title in [(axes[0], thr_default, f"Default threshold
    {thr_default:.2f}"),
                           (axes[1], thr_opt, f"Tuned threshold {thr_opt:.2f}")]:
        cm = confusion_matrix(y_test, (np.asarray(proba) >= thr).astype(int),
    labels=[0, 1])
        ax.imshow(cm, cmap=SEQ_CMAP, vmin=0, vmax=cm.max())
        for (i, j), v in np.ndenumerate(cm):
            ax.text(j, i, f"{v:,}", ha="center", va="center", fontsize=17,
    fontweight="bold",
                    color="white" if v > cm.max() * 0.5 else INK)
        ax.set_xticks([0, 1], CLASS_NAMES)
        ax.set_yticks([0, 1], CLASS_NAMES)
        ax.set(xlabel="Predicted", ylabel="Actual", title=title)
        ax.grid(False)
        for s in ax.spines.values():
            s.set_visible(False)
    fig.suptitle(f"Test-set confusion matrices - {title_suffix}", x=0.02, ha="left",
                 fontsize=12, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    plt.close(fig)


# ---------------------------------------------------------------------------- #
# Interpretability
# ---------------------------------------------------------------------------- #
def positive_class_shap(values) -> np.ndarray:
    """Normalise the different SHAP output layouts to (n_samples, n_features)."""
    if isinstance(values, list):  # older shap: one array per class
        values = values[1]
    values = np.asarray(values)
    return values[:, :, 1] if values.ndim == 3 else values


def shap_analysis(pipe: Pipeline, X_train, X_test, out_dir: Path, feature_set: str,
                  title_suffix: str) -> pd.DataFrame:
```

```
    import shap

    prep, scaler, clf = (pipe.named_steps[k] for k in ("prep", "scaler", "clf"))
    feats = model_features(feature_set)
    Xtr = scaler.transform(prep.transform(X_train.sample(min(SHAP_SAMPLE,
    len(X_train)), random_state=SEED)))
    Xte = scaler.transform(prep.transform(X_test.sample(min(SHAP_SAMPLE,
    len(X_test)), random_state=SEED)))

    if isinstance(clf, LogisticRegression):
        explainer = shap.LinearExplainer(clf, Xtr)
    else:
        explainer = shap.TreeExplainer(clf)
    sv = positive_class_shap(explainer.shap_values(Xte))

    # Group derived features under the measurement they come from (clinical reading)
    sv_df = pd.DataFrame(sv, columns=feats)
    grouped = sv_df.T.groupby(lambda f: FEATURE_GROUPS.get(f, f)).sum().T
    importance = (grouped.abs().mean().rename("mean_abs_shap").rename_axis("factor")
                  .sort_values(ascending=False).reset_index())

    top = importance.head(10).iloc[::-1]
    fig, ax = plt.subplots(figsize=(8, 4.8))
    ax.barh(top["factor"], top["mean_abs_shap"], color="#2a78d6", height=0.6)
    # scikit-learn forests explain probability; logistic regression / XGBoost
    explain log-odds
    unit = "probability" if isinstance(clf, RandomForestClassifier) else "log-odds"
    ax.set(xlabel=f"Mean |SHAP value| (impact on {unit} of diabetes)",
           title=f"What drives high-risk predictions - {title_suffix}")
    ax.grid(axis="y", visible=False)
    fig.tight_layout()
    fig.savefig(out_dir / "shap_importance_bar.png", dpi=160)
    plt.close(fig)

    plt.figure(figsize=(9, 6.5))
    shap.summary_plot(sv, Xte, feature_names=feats, show=False, max_display=14)
    plt.title(f"SHAP summary - {title_suffix} (red = high feature value)",
    loc="left")
    plt.tight_layout()
    plt.savefig(out_dir / "shap_beeswarm.png", dpi=160)
    plt.close()

    if hasattr(clf, "feature_importances_"):
        fi = pd.Series(clf.feature_importances_,
    index=feats).sort_values(ascending=False)
        print("\nBuilt-in tree feature importances (top 10):")
        print(fi.head(10).round(4).to_string())
    elif hasattr(clf, "coef_"):
        coefs = pd.Series(clf.coef_[0], index=feats).sort_values(key=np.abs,
    ascending=False)
        print("\nLogistic-regression coefficients on standardised features (top
    10):")
        print(coefs.head(10).round(4).to_string())
    return importance


# ------------------------------------------------------------------------ #
# One full run for one feature set
# ------------------------------------------------------------------------ #
def run_feature_set(feature_set, X_train, X_test, y_train, y_test, sex_test, args,
```

```
clean_log):
suffix = SET_TITLES[feature_set]
models_dir = Path(args.models_dir) / feature_set
out_dir = Path(args.outputs_dir) / feature_set
models_dir.mkdir(parents=True, exist_ok=True)
out_dir.mkdir(parents=True, exist_ok=True)

section(f"[{feature_set.upper()}] {suffix} - preprocessing parameters (training
set only)")
probe = ClinicalPreprocessor(feature_set=feature_set).fit(X_train)
params = pd.DataFrame({"median": probe.medians_})
params["clip_low"], params["clip_high"] = probe.lower_, probe.upper_
params["n_clipped"] = probe.count_clipped(X_train)
print(params.round(2).to_string())
print(f"Model features ({len(model_features(feature_set))}): {',
'.join(model_features(feature_set))}")

section(f"[{feature_set.upper()}] 5-fold stratified CV benchmark (training
set)")
cv_df, oof_probas = cv_benchmark(build_models(), X_train, y_train, feature_set)
display = cv_df[["model"]].copy()
for key in ["roc_auc", "pr_auc", "recall", "precision", "f1", "accuracy"]:
    display[key] = cv_df.apply(lambda r, k=key: f"{r[f'{k}_mean']:.3f} ±
{r[f'{k}_std']:.3f}", axis=1)
print("\n" + display.to_string(index=False))
cv_df.round(4).to_csv(out_dir / "cv_results.csv", index=False)
best_name = cv_df.loc[cv_df["roc_auc_mean"].idxmax(), "model"]
print(f"\nBest model by mean CV ROC-AUC: {best_name}")
plot_model_comparison(cv_df, out_dir / "model_comparison.png", suffix)

section(f"[{feature_set.upper()}] Clinical threshold tuning (target recall >=
{args.recall_target:.0%})")
oof_best = oof_probas[best_name]
thr_opt = select_threshold(y_train, oof_best, args.recall_target)
for label, thr in [("0.50", DEFAULT_THRESHOLD), (f"{thr_opt:.3f}", thr_opt)]:
    m = metrics_at_threshold(y_train, oof_best, thr)
    print(f"Out-of-fold @ {label:<6}: recall {m['recall']:.3f}, precision
{m['precision']:.3f}, "
          f"specificity {m['specificity']:.3f}")
sweep = pd.DataFrame([metrics_at_threshold(y_train, oof_best, t)
                      for t in np.round(np.arange(0.05, 0.71, 0.05), 2)])
sweep.round(3).to_csv(out_dir / "threshold_sweep.csv", index=False)
print("\nThreshold sweep (out-of-fold, training set):")
print(sweep[["threshold", "recall", "specificity", "precision",
"f1"]].round(3).to_string(index=False))
plot_threshold_tuning(y_train, oof_best, thr_opt, args.recall_target,
                      out_dir / "threshold_tuning.png", suffix)

section(f"[{feature_set.upper()}] Held-out test set evaluation")
fitted, test_probas = {}, {}
for name, model in build_models().items():
    fitted[name] = make_pipeline(model, feature_set).fit(X_train, y_train)
    test_probas[name] = fitted[name].predict_proba(X_test)[:, 1]
print("Test ROC-AUC / PR-AUC (reference only - selection used CV):")
for name, p in test_probas.items():
    print(f"  {name:<20} ROC-AUC {roc_auc_score(y_test, p):.3f}   PR-AUC
{average_precision_score(y_test, p):.3f}")

best_pipe, best_proba = fitted[best_name], test_probas[best_name]
print(f"\nSelected model: {best_name}")
```

```
m_default = print_threshold_report("DEFAULT decision threshold", y_test,
best_proba, DEFAULT_THRESHOLD)
m_tuned = print_threshold_report("OPTIMAL (recall-targeted) decision threshold",
y_test, best_proba, thr_opt)
r_ci, p_ci = bootstrap_ci(y_test, best_proba, thr_opt)
print(f"95% bootstrap CI at tuned threshold -> recall
{r_ci[0]:.3f}-{r_ci[1]:.3f}, "
      f"precision {p_ci[0]:.3f}-{p_ci[1]:.3f}")

# Does the model serve women and men equally well? (same threshold for everyone)
by_sex = {}
print("\nPerformance by sex at the tuned threshold:")
for sex in ["Female", "Male"]:
    mask = (sex_test == sex).to_numpy()
    m = metrics_at_threshold(y_test[mask], best_proba[mask], thr_opt)
    m["roc_auc"] = float(roc_auc_score(y_test[mask], best_proba[mask]))
    m["n"], m["n_diabetic"] = int(mask.sum()), int(y_test[mask].sum())
    by_sex[sex] = m
    print(f"  {sex:<6} n={m['n']:>5} (diabetic {m['n_diabetic']:>4})  ROC-AUC
{m['roc_auc']:.3f}  "
          f"recall {m['recall']:.3f}  precision {m['precision']:.3f}
specificity {m['specificity']:.3f}")

plot_roc_pr(y_test, test_probas, best_name, thr_opt, out_dir /
"roc_pr_curves.png",
            args.recall_target, suffix)
plot_confusions(y_test, best_proba, DEFAULT_THRESHOLD, thr_opt,
                out_dir / "confusion_matrices.png", suffix)

section(f"[{feature_set.upper()}] Interpretability (SHAP)")
importance = shap_analysis(best_pipe, X_train, X_test, out_dir, feature_set,
suffix)
importance.round(4).to_csv(out_dir / "shap_importance.csv", index=False)
print("\nTop clinical factors by mean |SHAP| (derived features grouped with
their base measurement):")
print(importance.head(8).round(4).to_string(index=False))

prep = best_pipe.named_steps["prep"]
joblib.dump(best_pipe.named_steps["clf"], models_dir / "best_model.joblib")
joblib.dump(best_pipe.named_steps["scaler"], models_dir / "scaler.joblib")
joblib.dump(prep.get_params_dict(), models_dir / "imputation_params.joblib")
joblib.dump(best_pipe, models_dir / "full_pipeline.joblib")  # convenience for
batch scoring
best_cv = cv_df[cv_df["model"] == best_name].iloc[0]
metadata = {
    "feature_set": feature_set,
    "feature_set_description": suffix,
    "model_name": best_name,
    "decision_threshold": thr_opt,
    "default_threshold": DEFAULT_THRESHOLD,
    "recall_target": args.recall_target,
    "raw_columns": prep.get_params_dict()["raw_columns"],
    "model_features": model_features(feature_set),
    "cv_roc_auc_mean": float(best_cv["roc_auc_mean"]),
    "test_roc_auc": float(roc_auc_score(y_test, best_proba)),
    "test_pr_auc": float(average_precision_score(y_test, best_proba)),
    "test_metrics_default": m_default,
    "test_metrics_tuned": m_tuned,
    "test_recall_ci95": [float(r_ci[0]), float(r_ci[1])],
    "test_precision_ci95": [float(p_ci[0]), float(p_ci[1])],
```

```
        "test_metrics_by_sex_tuned": by_sex,
        "n_train": int(len(X_train)), "n_test": int(len(X_test)),
        "positive_rate": float(np.mean(np.r_[y_train, y_test])),
        "cleaning_log": clean_log,
        "trained_at_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "sklearn_version": sklearn.__version__,
        "random_state": SEED,
    }
    (models_dir / "model_metadata.json").write_text(json.dumps(metadata, indent=2))
    print(f"\nSaved artifacts to {models_dir.resolve()}: "
          + ", ".join(sorted(f.name for f in models_dir.iterdir())))
    return metadata


# ---------------------------------------------------------------------------- #
# Main
# ---------------------------------------------------------------------------- #
def main() -> None:
    parser = argparse.ArgumentParser(description="Train the both-sexes early
    diabetes prediction models.")
    parser.add_argument("--data", default="data/diabetes_prediction_dataset.csv")
    parser.add_argument("--models-dir", default="models")
    parser.add_argument("--outputs-dir", default="outputs")
    parser.add_argument("--feature-sets", nargs="+", default=list(FEATURE_SETS),
    choices=FEATURE_SETS)
    parser.add_argument("--recall-target", type=float, default=0.85)
    args = parser.parse_args()

    section("1. DATA LOADING, CLEANING AND AUDIT")
    df, clean_log = load_data(args.data)
    print("Cleaning log (rows remaining after each step):")
    for k, v in clean_log.items():
        print(f"  {k:<30} {v:>8,}")
    print(f"\nRows: {len(df):,}   diabetic: {df[TARGET].sum():,}
    ({df[TARGET].mean():.1%})")
    by_sex = df.groupby("gender")[TARGET].agg(rows="count", diabetic="sum",
    rate="mean")
    print("\nBy sex:\n" + by_sex.round(3).to_string())
    bmi_placeholder = (df["bmi"].round(2) == 27.32).mean()
    print(f"\nBMI == 27.32 (upstream placeholder, treated as missing):
    {bmi_placeholder:.1%} of rows")

    section("2. STRATIFIED 80/20 TRAIN-TEST SPLIT")
    X, y = df[ALL_RAW_COLUMNS], df[TARGET].to_numpy()
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, stratify=y, random_state=SEED
    )
    print(f"Train: {len(X_train):,} rows ({y_train.mean():.1%} diabetic) | "
          f"Test: {len(X_test):,} rows ({y_test.mean():.1%} diabetic)")
    print(f"Test set sex mix: {X_test['gender'].value_counts().to_dict()}")

    results = {}
    for fs in args.feature_sets:
        results[fs] = run_feature_set(fs, X_train, X_test, y_train, y_test,
                                      X_test["gender"], args, clean_log)

    section("SUMMARY")
    rows = []
    for fs, md in results.items():
        t = md["test_metrics_tuned"]
```

```
        rows.append({
            "feature set": fs, "model": md["model_name"], "threshold":
    round(md["decision_threshold"], 3),
            "test ROC-AUC": round(md["test_roc_auc"], 3), "recall":
    round(t["recall"], 3),
            "precision": round(t["precision"], 3), "specificity":
    round(t["specificity"], 3),
            "recall F / M":
    f"{md['test_metrics_by_sex_tuned']['Female']['recall']:.2f} / "
f"{md['test_metrics_by_sex_tuned']['Male']['recall']:.2f}",
        })
    print(pd.DataFrame(rows).to_string(index=False))
    print(f"\nFigures: {Path(args.outputs_dir).resolve()}   Models:
    {Path(args.models_dir).resolve()}")
    print("Launch the app with:  streamlit run app.py")


if __name__ == "__main__":
    main()
```